LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

CS4001NI Programming

60% Individual Coursework

2025 Spring

Student Name: Priya Thapa

London Met ID: 24046447

College ID: NP01NT4A240101

Group: L1N1

Assignment Due Date: Friday, May 16, 2025

Assignment Submission Date: Friday, May 16, 2025

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

11.docx

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:96194736

Submission Date
May 16, 2025, 12:27 PM GMT+5:45

Download Date
May 16, 2025, 12:29 PM GMT+5:45

File Name
11.docx

File Size
59.7 KB

61 Pages

6,084 Words

40,103 Characters

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

70 Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

21 Missing Citation 3%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
14% Submitted works (Student Pap

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a d
would set it apart from a normal submissio
it for you to review.

A Flag is not necessarily an indicator of a p
focus your attention there for further revie

Table of Contents

Table of figures

Table of Tables

1. Introduction

In today's fast-paced world, fitness and gym memberships have become an essential part of a healthy lifestyle. Managing gym members efficiently is crucial to ensuring a seamless experience for both the gym administration and its members. This project involves building a java object-oriented programming-based system to manage gym members. We have a parent class named Gym member which contains common data for all the members such as personal details, membership start date and attendance. The base class is extended by two child classes named Regular Member and Premium Member, which have specific functionality like payment processing, loyal points and various memberships plan.

1.1 Aims and Objective

The primary goal of this project is to implement a real-world application using object-oriented programming. The system will store each member's information in an Array list and enable the tracking of loyal points and attendance. GUI will be used to interact with the system and ease the user to manage and view the members' information.

The key objectives of this project include:

- Implementing OOP concepts such as inheritance, encapsulation, and polymorphism in Java.
- Developing a structured class hierarchy, where Gym Member serves as the base class, and Regular Member and Premium Member servers as the subclass.
- Tracking gym attendance and loyalty points for each member.
- Handling membership plans by managing payments, upgrades, and special benefits. Integrating a GUI to provide a user-friendly interface for managing gym members

2. Wireframe

A wireframe is a visual sketch of an application or website's underlying architecture. It assists designers and developers in planning the user flow and structure efficiently, with emphasis on layout and function rather than design elements like colour or graphics. Wireframes are essential in the initial stages of development to define usability problems, communicate ideas, and obtain stakeholder commitment (Interaction Design Foundation, 2024).

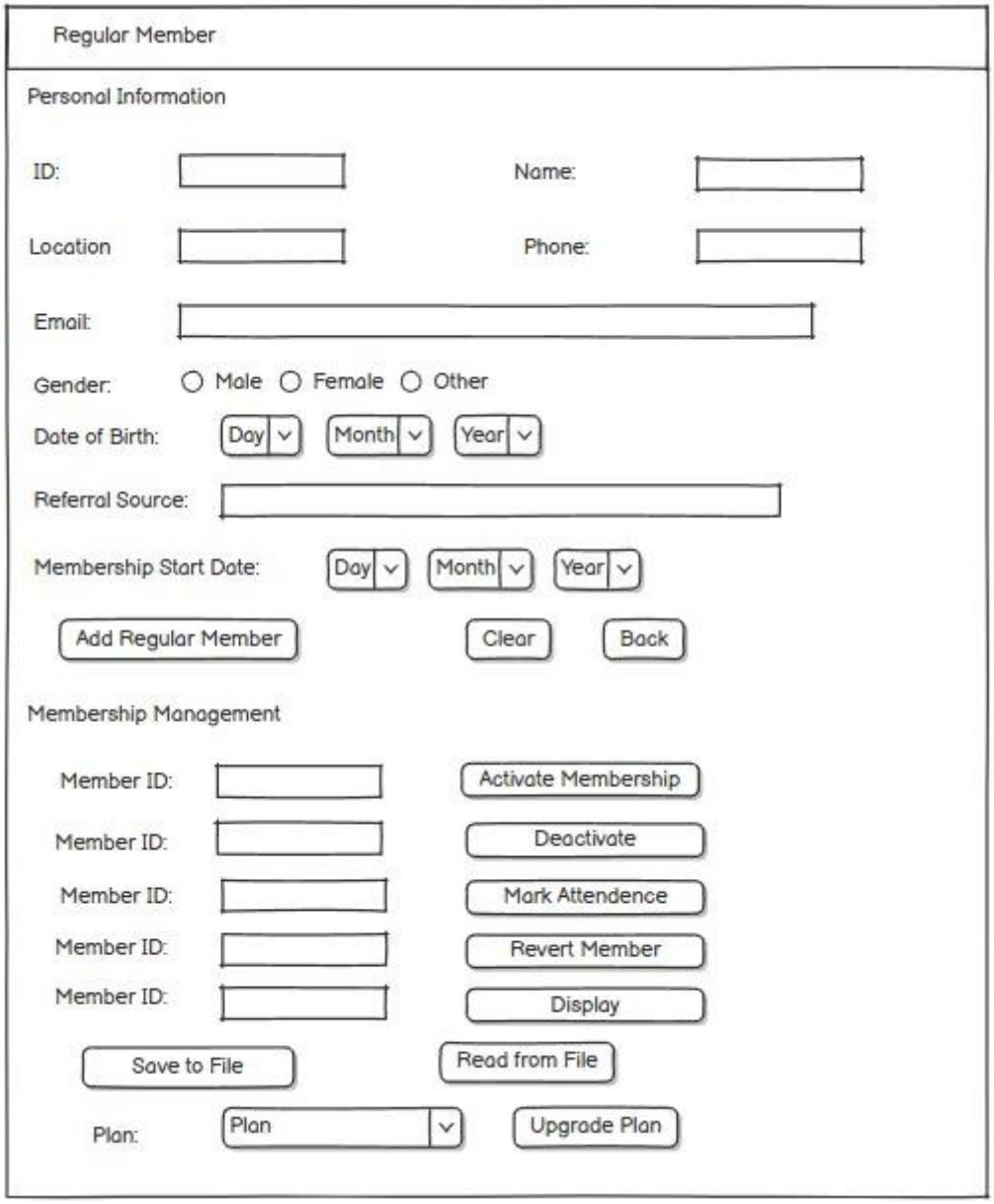

Figure 1 Regular Member wireframe

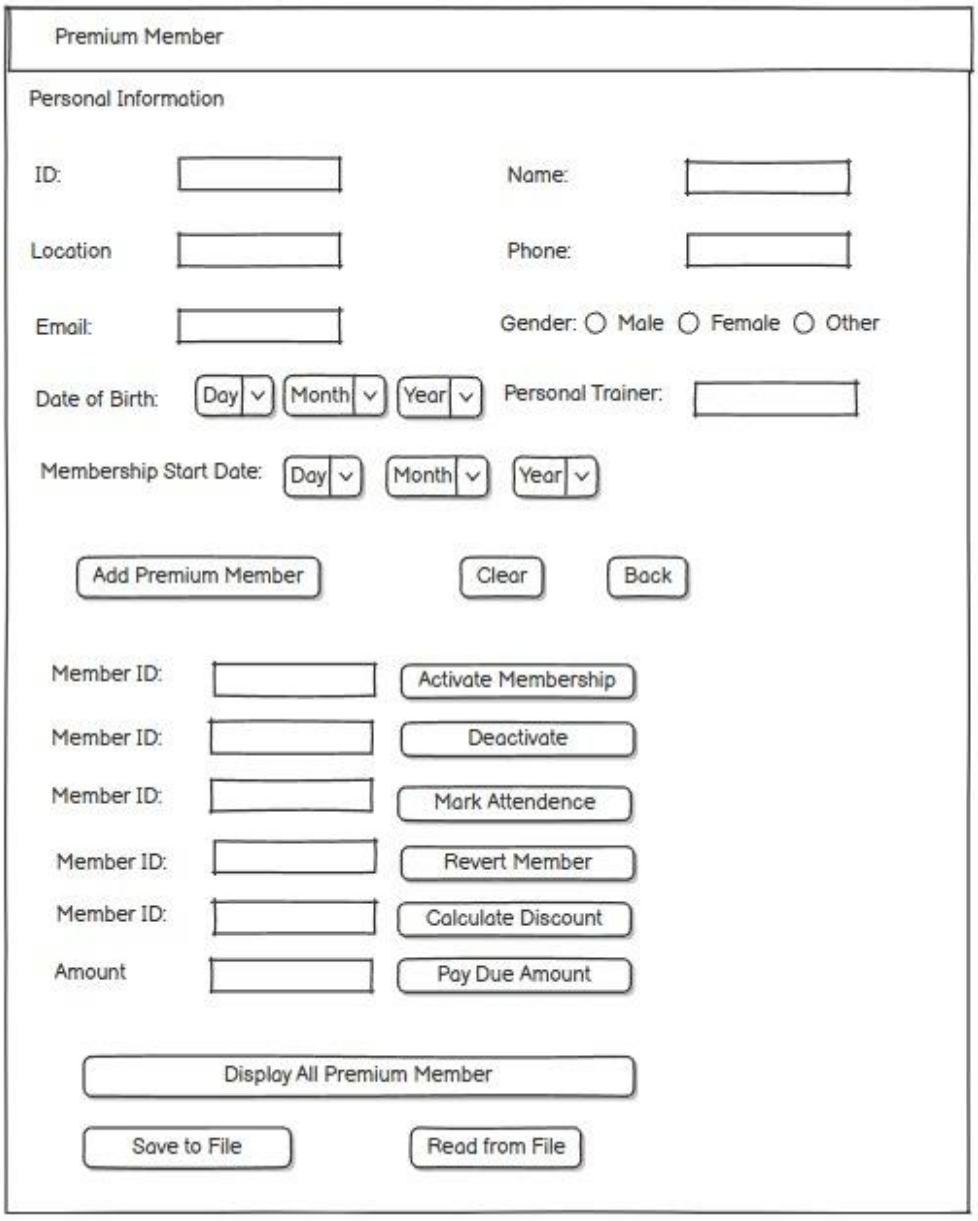

Figure 2 Premium Member wireframe

3.Class Diagram

Each class, with all its attributes and operations, along with its various connections to other classes, are all shown. Class diagrams are key elements of the development process because they let designers draw, visualize, and document a software system.

How a program carries out the function or process it was created for is described in the method description. Most of the time, the method name, its arguments, type of result, and a short explanation of its role are listed. Good descriptions of how methods work help developers see where different parts of the code interact and are very important for code readability, upkeep, and sharing tasks (Lucichart, 2024).

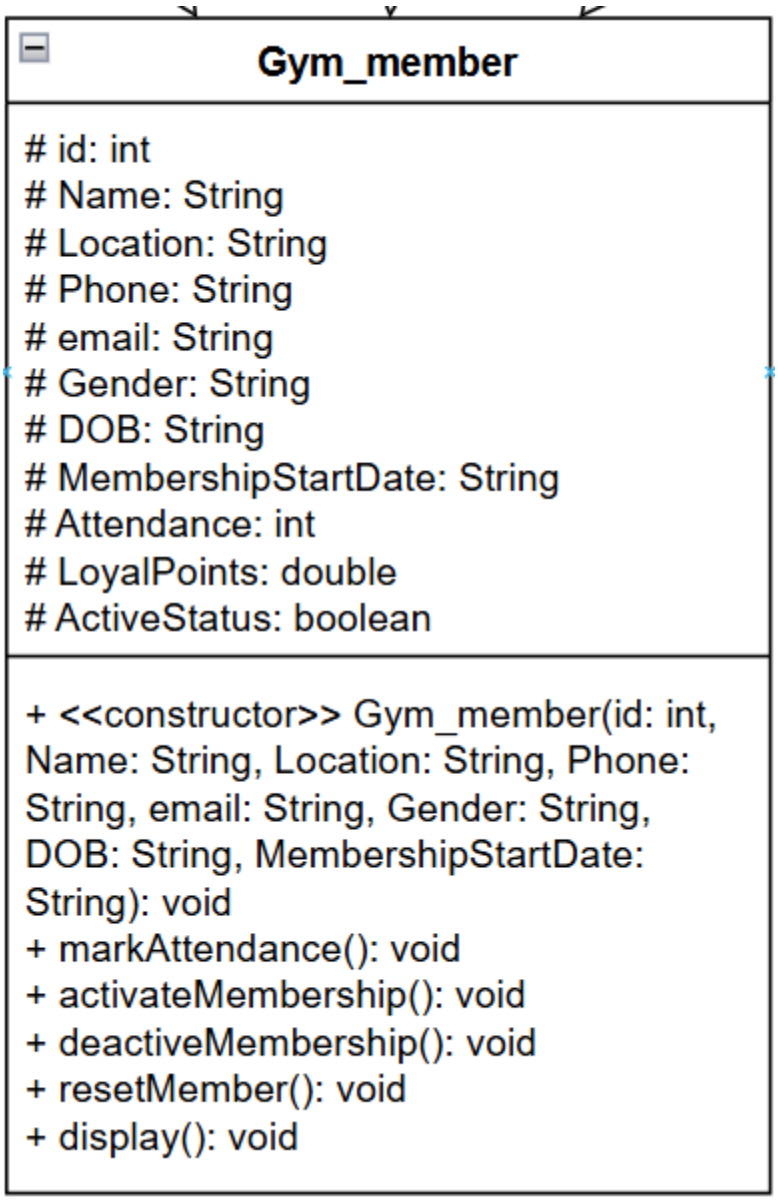

Figure 3Class diagram of Gym Member

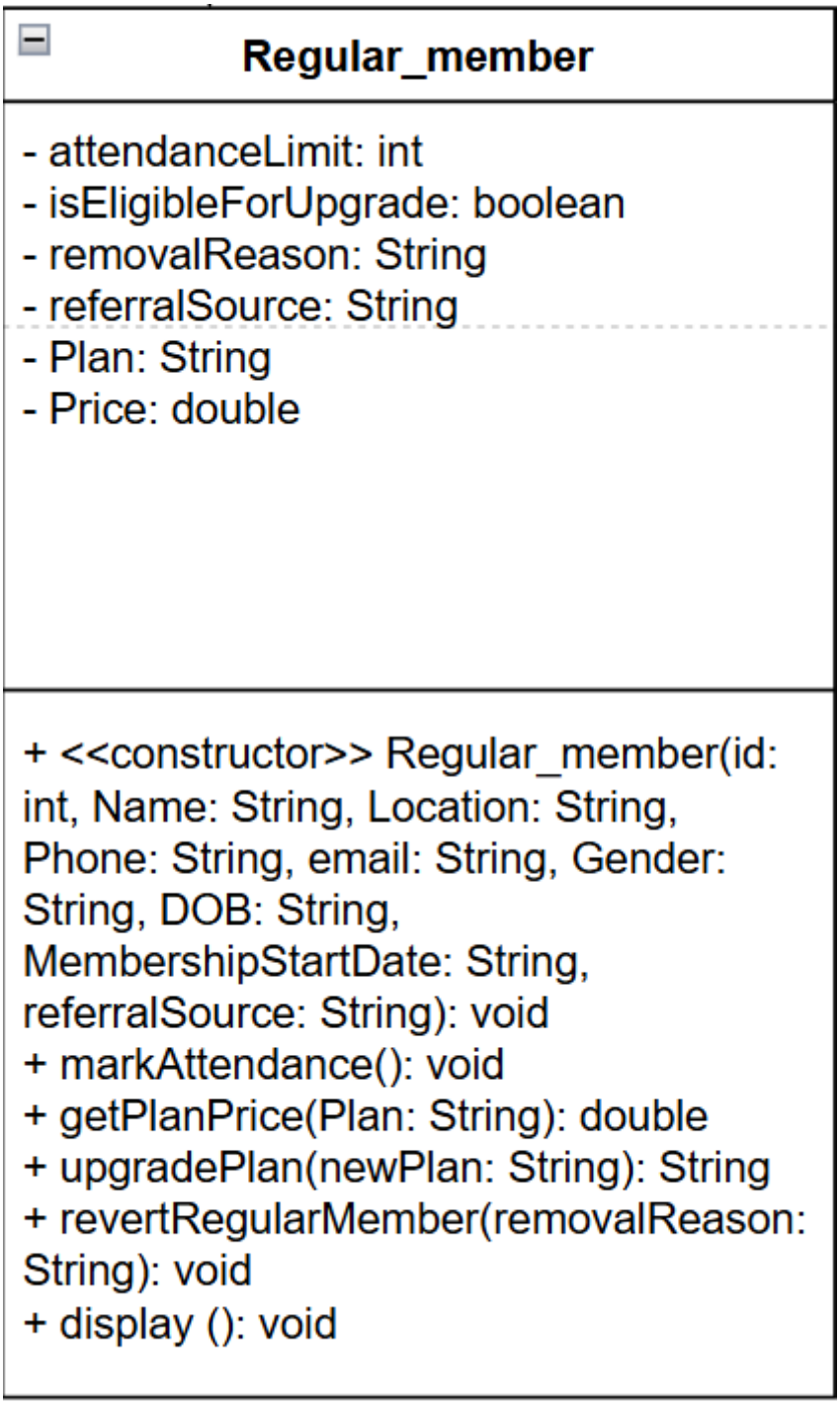

Figure 4Class diagram of Regular Member

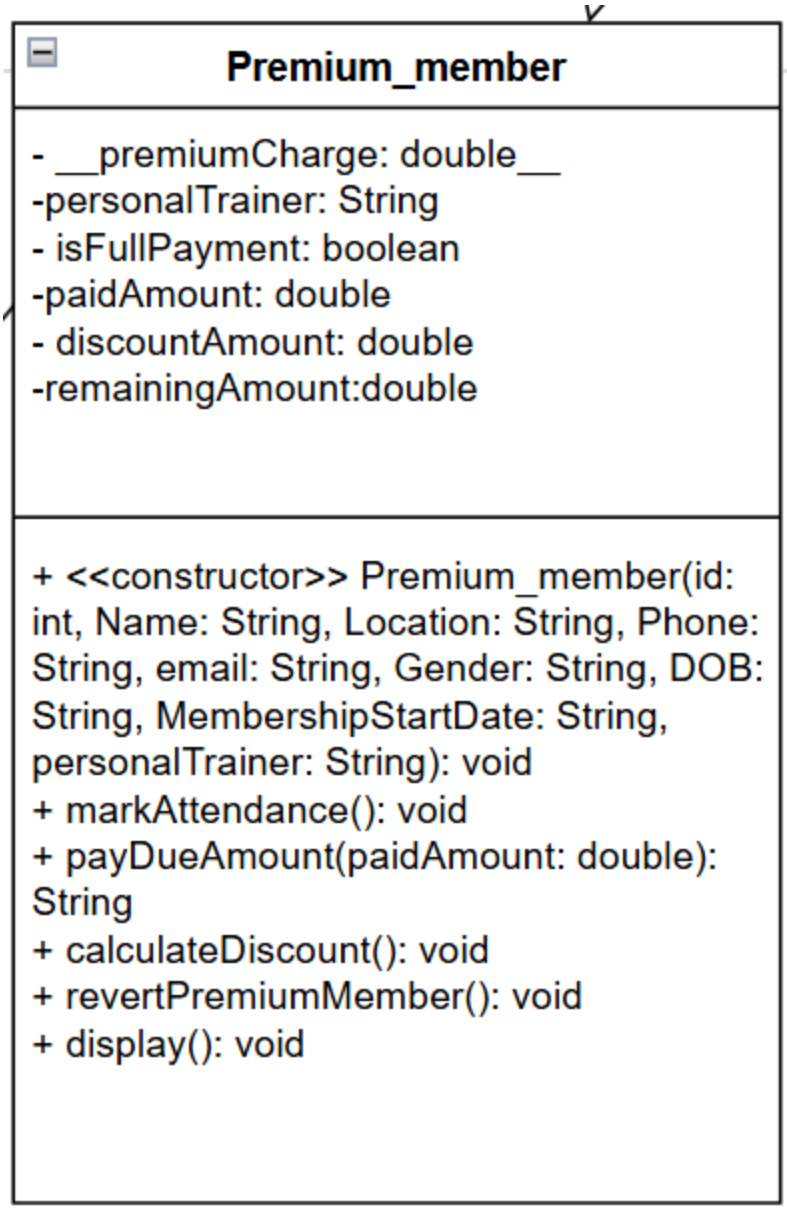

Figure 5Class diagram of Premium Member

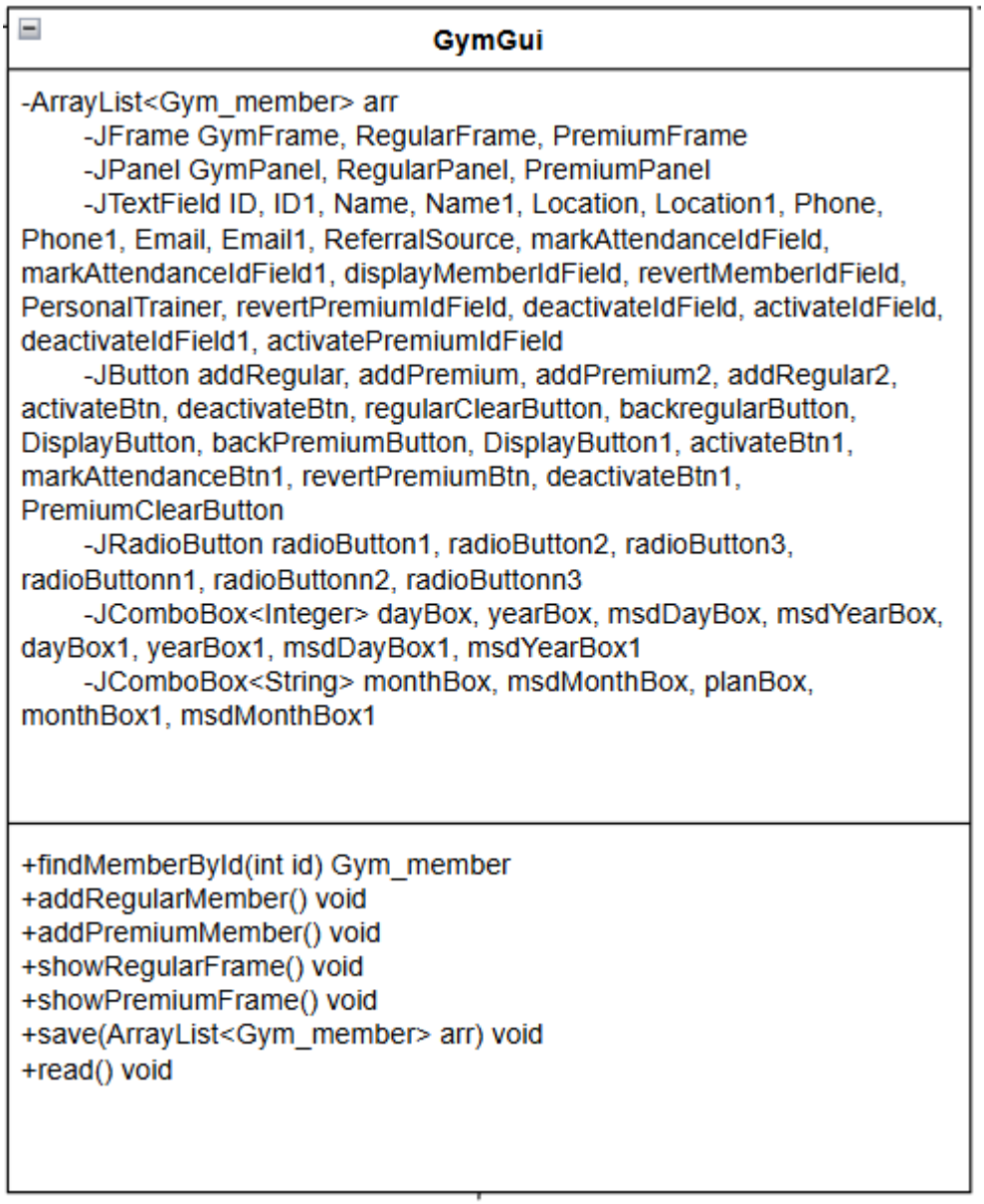

Figure 6Class diagram of Gym GUI

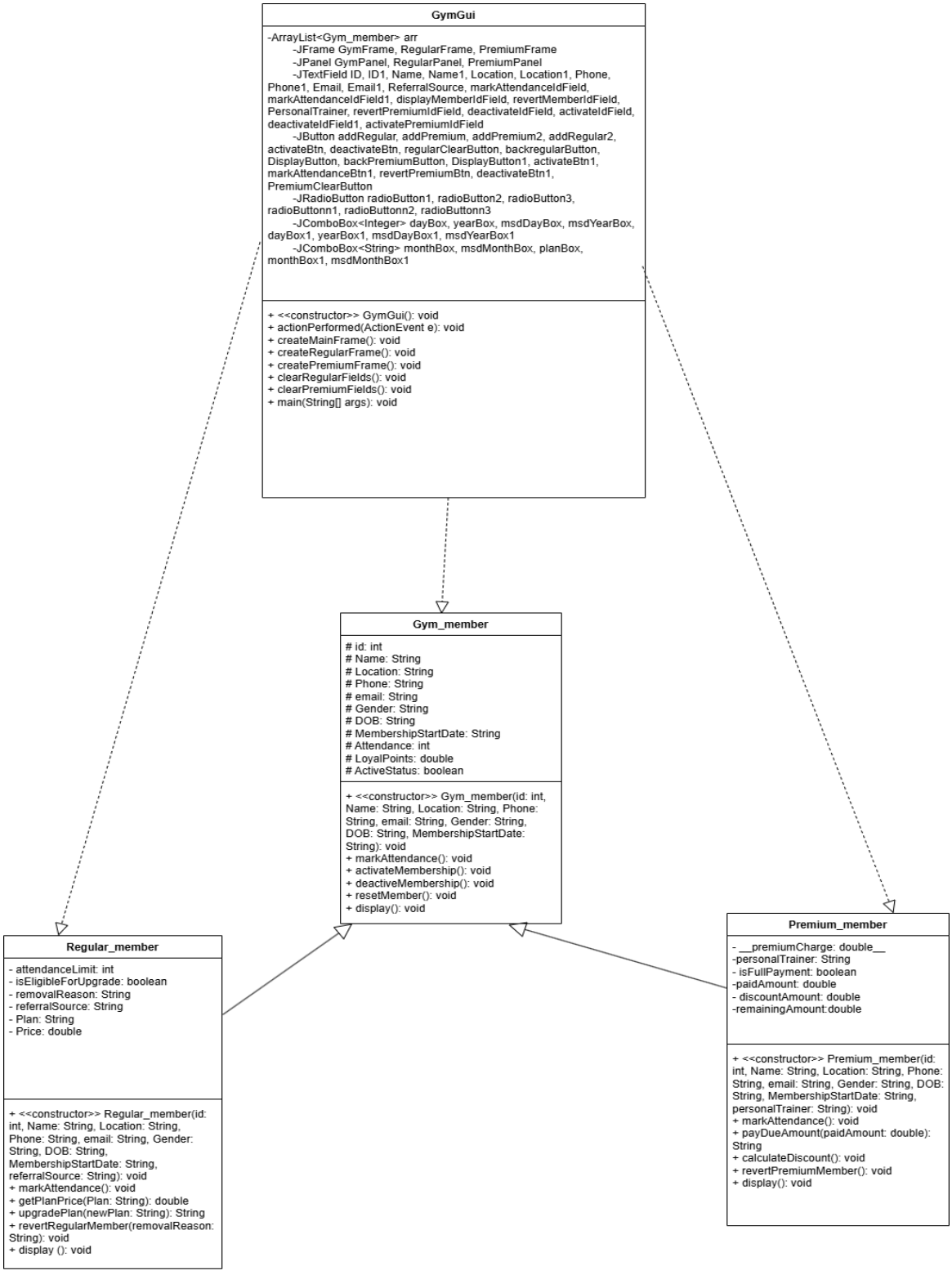

Figure 7 Entire class diagram

4. PSEUDOCODE

A way of writing computer logic in plain language, following the form of real code, is called pseudocode. Because engineers and newcomers can develop algorithms and follow program logic freely, it helps with early problem-solving and is not hard to switch to a programming language during implementation (GeeksforGeeks, 2021).

Pseudocode of Gym Member

CREATE class Gym_member

DO

DECLARE instance variable id as int using protected access modifier

DECLARE instance variable Name as String using protected access modifier

DECLARE instance variable Location as String using protected access modifier

DECLARE instance variable Phone as String using protected access modifier

DECLARE instance variable email as String using protected access modifier

DECLARE instance variable Gender as String using protected access modifier

DECLARE instance variable DOB as String using protected access modifier

DECLARE instance variable MembershipStartDate as String using protected access modifier

DECLARE instance variable Attendance as int using protected access modifier

DECLARE instance variable LoyalPoints as double using protected access modifier

DECLARE instance variable ActiveStatus as int using protected access modifier

CREATE constructor Gym_member with parameter id1, Name1, Location1, Phone1, email1, Gender1, String DOB1, MembershipStartDate1

DO

ASSIGN instance variable of Attendance to 0

ASSIGN instance variable of ActiveStatus to false

ASSIGN instance variable of LoyalPoint to 0.0

ASSIGN instance variable of id to parameter value of id1

ASSIGN instance variable of Name to parameter value of Name1

ASSIGN instance variable of Location to parameter value of Location1

ASSIGN instance variable of Phone to parameter value of Phone1

ASSIGN instance variable of email to parameter value of email1

ASSIGN instance variable of Gender to parameter value of Gender1

ASSIGN instance variable of DOB to parameter value of DOB1

ASSIGN instance variable of MembershipStartDate to parameter value of MembershipStartDate1

END DO

CREATE abstract method name markAttendance

CREATE method name activeMembership

DO

IF ActiveStatus is FALSE then

SET ActiveStatus to true

DISPLAY "The membership [id] is successfully activated"

ELSE

DISPLAY "The membership [id] is already activated"

END IF

END DO

CREATE method name deactiveMembership

DO

IF ActiveStatus is TRUE then

SET ActiveStatus to FALSE

DISPLAY "The membership [id] is successfully deactivated"

ELSE

DISPLAY "The membership [id] is already deactivated"

END IF

END DO

CREATE method name resetMember

DO

SET ActiveStatus to FALSE

SET Attendance to 0

SET LoyalPoints to 0.0

DISPLAY "Membership [id] is reset successfully"

```
        END DO
    CREATE method named display


        DO
            DISPLAY "Member ID: " + id
            DISPLAY "Member Name: " + name
            DISPLAY "Member Location: " + Location
            DISPLAY "Member Phone Number: " + Phone
            DISPLAY "Member Email: " + email
           DISPLAY "Gender of Member: " + Gender
           DISPLAY "Date of Birth of Member: " + DOB
          DISPLAY "Membership Start Date: " + MembershipStartDate
          DISPLAY "Membership Attendance: " + Attendance
          DISPLAY "Membership Loyal Points: " + LoyalPoints
          DISPLAY "Membership Active Status: " + ActiveStatus
        END DO
END DO
```

Pseudocode of Regular Member

CREATE class Regular_member (Child class)

DO

DECLARE Constant variable attendanceLimit as int using Private access modifier

DECLARE instance variable isEligibleForUpgrade as Boolean using Private access modifier

DECLARE instance variable removalReason as String using private access modifier

DECLARE instance variable referralSource as String using private access modifier

DECLARE instance variable Plan as String using private access modifier

DECLARE instance variable Price as double using private access modifier

CREATE constructor Regular_member with parameter id1, Name1, Location1, Phone1, email1, Gender1, String DOB1, MembershipStartDate1, referralSource1

DO

CALL parent class constructor with (id1, Name1, Location1, Phone1, email1, Gender1, String DOB1, MembershipStartDate1)

ASSIGN instance variable of isEligibleForUpgrade to false

ASSIGN instance variable of attendanceLimit to 30

ASSIGN instance variable of Plan to Basic

ASSIGN instance variable of Price to 6500

ASSIGN instance variable of removalReason to empty

ASSIGN instance variable of referralSource to parameter value of referralSource1

END DO

CREATE method name markAttendance

DO

IF ActiveStatus is TRUE then

CALL attendance from parent class and add value by +1

CALL LoyalPoints from parent class and add value by +5

DISPLAY "Attendance marked for Regular Member: " + Name

DISPLAY "Total Attendance: " + Attendance

DISPLAY "Loyalty Points: " + LoyalPoints

ELSE

DISPLAY "Account is deactive"

END IF

END DO

CREATE method name getPlanPrice(input plan as a string) with return type double

DO

CONVERT plan to lowercase

SWITCH Plan

CASE basic

DISPLAY "Price of this " + Plan + " plan is 6500"

ASSIGN Price to 6500

RETURN Price

CASE standard

DISPLAY "Price of this " + Plan + " plan is 12500"

ASSIGN Price to 12500

RETURN Price

CASE deluxe

DISPLAY "Price of this " + Plan + " plan is 18500"

ASSIGN Price to18500

RETURN Price

DEFAULT:

RETURN -1

END SWITCH

END DO

CREATE method name upgradePlan (input newPlan as String)

DO

IF Attendance is greater than equals to attendanceLimit then

SET isEligibleForUpgrade to TRUE

ELSE

SET isEligibleForUpgrade to FALSE

END IF

IF isEligibleForUpgrade is FALSE then

RETURN "You are not eligible for an upgrade yet."

IF Plan equals newPlan then

RETURN "You are already subscribed to this plan."

END IF

SET Plan TO newPlan

SET newPrice TO getPlanPrice

IF newPrice = -1 THEN

RETURN "Invalid plan selected."

END IF

SET Price to newPrice

RETURN "Plan upgraded successfully to " + newPlan + " with price " + newPrice

END DO

CREATE method name revertRegularMember (input removalReason as a string)

DO

CALL parent class method name resetMember

SET isEligibleForUpgrade to FALSE

SET Plan to Basic

SET Price to 6500

SET removalReason TO input removalReason

END DO

CREATE method name display

DO

CALL superclass display

DISPLAY "Member ID: " + id

DISPLAY "Plan: " + Plan

DISPLAY "Price: " + Price

IF removalReason is not empty then

DISPLAY "Removal Reason: " + removalReason

```
            END IF

        END DO

END DO
```

Pseudocode of Premium Member

CREATE class Premium_member (Child class)

DO

DECLARE Constant variable premiumCharge as double using Private access modifier

DECLARE instance variable personalTrainer as String using Private access modifier

DECLARE instance variable isFullPayment as Boolean using private access modifier

DECLARE instance variable paidAmount as double using private access modifier

DECLARE instance variable discountAmount as double sing private access modifier

CREATE constructor Premium_member with parameter id1, Name1, Location1, Phone1, email1, Gender1, String DOB1, MembershipStartDate1, personalTrainer1

DO

CALL parent class constructor with (id1, Name1, Location1, Phone1, email1, Gender1, String DOB1, MembershipStartDate1)

ASSIGN instance variable of premiumCharge to 50000.0

ASSIGN instance variable of paidAmount to 0.0

ASSIGN instance variable of Plan to isFullPayment to false

ASSIGN instance variable of discountAmount to 0.0

ASSIGN instance variable of removalReason to empty

ASSIGN instance variable of personalTrainer to parameter value of personalTrainer1

END DO

CREATE method name markAttendance

DO

IF ActiveStatus is TRUE then

CALL attendance from parent class and add value by +1

CALL LoyalPoints from parent class and add value by +10

DISPLAY "Attendance marked for Premium Member: " + Name

DISPLAY "Total Attendance: " + Attendance

DISPLAY "Loyalty Points: " + LoyalPoints

ELSE

DISPLAY "Account is deactive"

END IF

END DO

CREATE method name payDueAmount (input paidAmount as a double) with return type String

DO

IF isFullPayment IS TRUE then

RETURN "Amount had been paid already"

ELSE IF premiumCharge < paidAmount then

SET isFullPayment TO TRUE

RETURN "Amount exceeds premium charge. Full payment recorded."

ELSE IF premiumCharge > paidAmount then

SET remainingAmount TO premiumCharge - paidAmount

SET isFullPayment to FALSE

RETURN "Remaining amount to pay: Rs. " + remainingAmount

ELSE

SET isFullPayment to TRUE

RETURN "Full payment received"

END DO

CREATE method name CalculateDiscount

DO

IF isFullPayment is true then

SET discountAmount to 0.1 × premiumCharge

DISPLAY "Your discount is: " + discountAmount

ELSE

DISPLAY "No discount"

END IF

END DO

CREATE method name revertPremiumMember

DO

CALL parent class method name resetMember

SET personalTrainer to empty

SET isFullPayment to FALSE

SET PaidAmount to 0.0

SET discountAmount to 0.0

END DO

CREATE method name display

DO

CALL superclass display

DISPLAY "Personal Trainer: " + personalTrainer

DISPLAY "Paid Amount: " + paidAmount

DISPLAY "Full Payment: " + isFullPayment

```
            SET remainingAmount TO premiumCharge - paidAmount

                DISPLAY "Remaining Amount: " + remainingAmount

            IF isFullPayment IS TRUE THEN

                DISPLAY "Discount Amount: " + discountAmount

            END IF

        END DO

    END DO
```

Pseudocode of Gym GUI

Initialize the main frame and panels for the application.

Initialize buttons for adding regular and premium members.

Initialize text fields and combo boxes for member details.

Initialize action listeners for buttons.

Main Function:

Initialize the main frame and set its properties.

Create buttons for adding regular and premium members.

Add action listeners to the buttons.

Display the main frame.

Add Regular Member:

Create a new frame for adding regular members.

Add text fields for member details (ID, Name, Location, Phone, Email, Gender, DOB, Membership Start Date, Referral Source).

Add combo boxes for selecting dates and plans.

Add buttons for adding the member, clearing fields, and going back.

Add action listeners to the buttons.

Display the regular member frame.

Add Premium Member:

Create a new frame for adding premium members.

Add text fields for member details (ID, Name, Location, Phone, Email, Gender, DOB, Membership Start Date, Personal Trainer).

Add combo boxes for selecting dates.

Add buttons for adding the member, clearing fields, and going back.

Add action listeners to the buttons.

Display the premium member frame.

Activate/Deactivate Membership:

Prompt for member ID.

Search for the member in the list.

If member found, activate or deactivate the membership.

Display a message indicating success or failure.

Mark Attendance:

Prompt for member ID.

Search for the member in the list.

If member found and membership is active, mark attendance.

Display a message with attendance details.

Revert Member:

Prompt for member ID.

Search for the member in the list.

If member found, revert the member's details to default values.

Display a message indicating success.

Upgrade Plan:

Prompt for member ID and new plan.

Search for the member in the list.

If member found and membership is active, upgrade the plan.

Display a message with the result.

Calculate Discount:

Prompt for member ID.

Search for the member in the list.

If member found and membership is active, calculate the discount.

Display a message with the discount amount.

Pay Due Amount:

Prompt for member ID and payment amount.

Search for the member in the list.

If member found and membership is active, process the payment.

Display a message with the payment status.

Save to File:

Open a file for writing.

Write member details to the file.

Display a message indicating success.

Read from File:

Open the file for reading.

Read member details from the file.

Display the details in a new frame.

Display Member Details:

Prompt for member ID.

Search for the member in the list.

If member found, display their details

5.Method Description

Class: Gym_member

This is an abstract class representing a gym member's details and membership status.

Methods:

Constructor

Initializes the member's details such as ID, name, location, phone, email, gender, date of birth, and membership start date.

Sets default values for attendance, loyalty points, and active status.

Abstract Method

markAttendance(): An abstract method that must be implemented by subclasses to mark attendance.

activateMembership()

Activates the membership if it is not already active.

Prints a success message if activated, otherwise indicates it is already active.

deactiveMembership()

Deactivates the membership if it is active.

Prints a success message if deactivated, otherwise indicates it is already deactivated.

resetMember()

Resets the member's data to default values (attendance, loyalty points, active status).

display()

Displays the member's details including ID, name, location, phone, email, gender, date of birth, membership start date, attendance, loyalty points, and active status.

Class: Regular_member

This class extends Gym_member and represents a regular gym member with additional features like attendance tracking, loyalty points, and plan upgrades.

Methods:

Constructor

Initializes the member's details and sets default values for attendance limit, plan, price, and referral source.

Getter Methods

markAttendance()

Marks attendance for the member if the membership is active.

Increases attendance count and loyalty points.

getPlanPrice(String Plan)

Returns the price of the selected plan (Basic, Standard, Deluxe).

upgradePlan(String newPlan)

Upgrades the member's plan if they are eligible (based on attendance).

Updates the plan and price accordingly.

revertRegularMember(String removalReason)

Reverts the member's details to default values and sets the removal reason.

display()

Displays the member's details including plan and price.

If a removal reason is set, it is also displayed.

Class: Premium_member

This class extends Gym_member and represents a premium gym member with additional features like a personal trainer, full payment status, and premium charges.

Methods:

Constructor

Initializes the member's details and sets default values for premium charge, paid amount, full payment status, and discount amount.

Getter Methods

markAttendance()

Marks attendance for the member if the membership is active.

Increases attendance count and loyalty points.

payDueAmount(double paidAmount)

Handles payment of dues and checks full payment status.

Returns a message indicating the payment status.

calculateDiscount()

Calculates a discount if the member has made a full payment.

revertPremiumMember()

Reverts the member's details to default values.

display()

Displays the member's details including personal trainer, paid amount, full payment status, and discount amount.

Class: Gym_GUI

This class represents the graphical user interface for managing gym members.

Methods:

Constructor

Initializes the GUI components including frames, panels, buttons, text fields, and combo boxes.

Sets up the main frame, regular member frame, and premium member frame.

showRegularFrame()

Displays the regular member frame.

showPremiumFrame()

Displays the premium member frame.

addRegularMember()

Adds a new regular member to the list.

Validates input fields and checks for duplicate IDs.

addPremiumMember()

Adds a new premium member to the list.

Validates input fields and checks for duplicate IDs.

save(ArrayList<Gym_member> arr)

Saves the member list to a file.

read()

Reads the member list from a file and displays it in a new frame.

findMemberById(int id)

Finds a member by ID in the list.

6.Testing

Testing is the process of running a software application to find and address errors or problems. Software needs to be as free of errors as possible to run properly and consistently. This procedure improves the user experience in addition to verifying the program runs as planned. Furthermore, testing provides useful feedback that guides upcoming improvements and updates.

6.1Testing1 command prompt

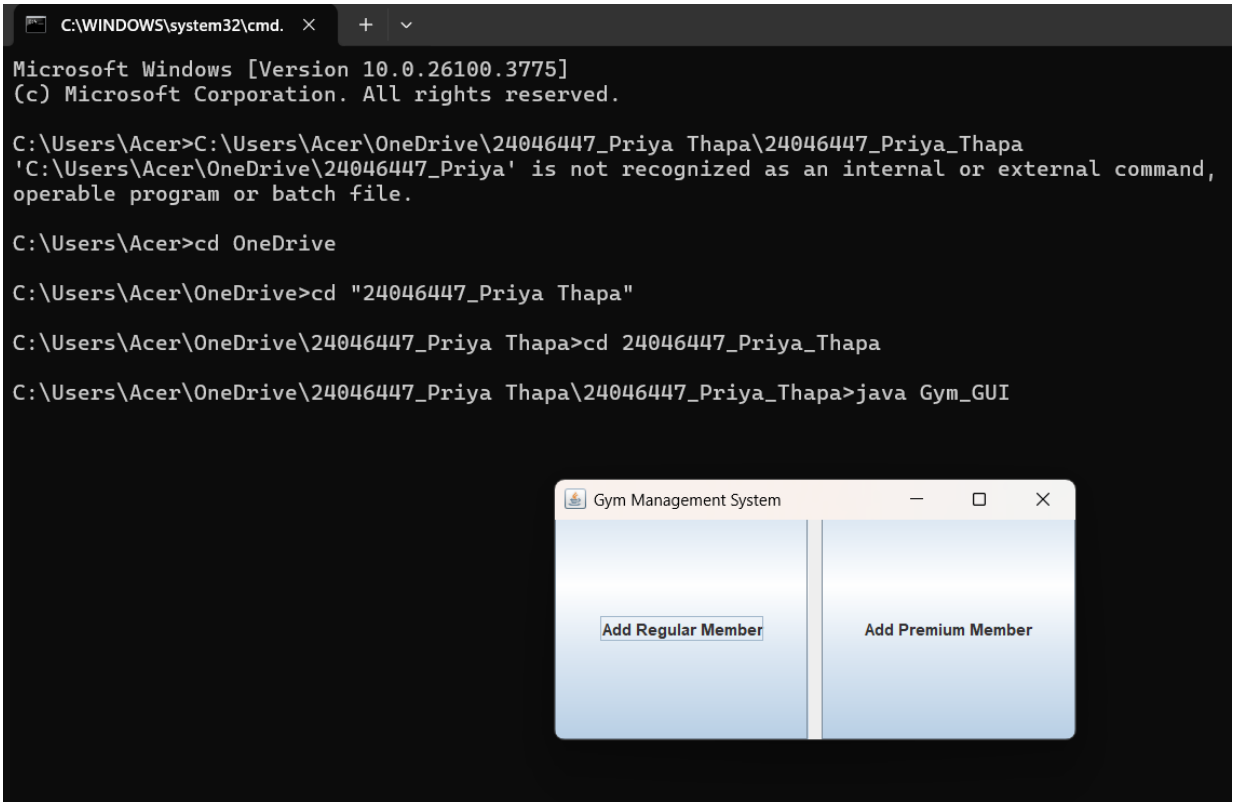

Figure 8 Testing1.comand prompt

Objective	To verify the program can be compiled and executed successfully via command line.
Action	1. Open command prompt/Terminal. 2. Navigate to the project directory. 3. Compile the program using java Gym_GUI.
Expected Output	The expected output should be displayed when the program executes successfully and compiles without any issues.
Actual Output	The program compiles and runs without error, displaying the correct output as expected.
Result	The test was successful.

Table 1 .Command prompt testing

6.2Testing 2

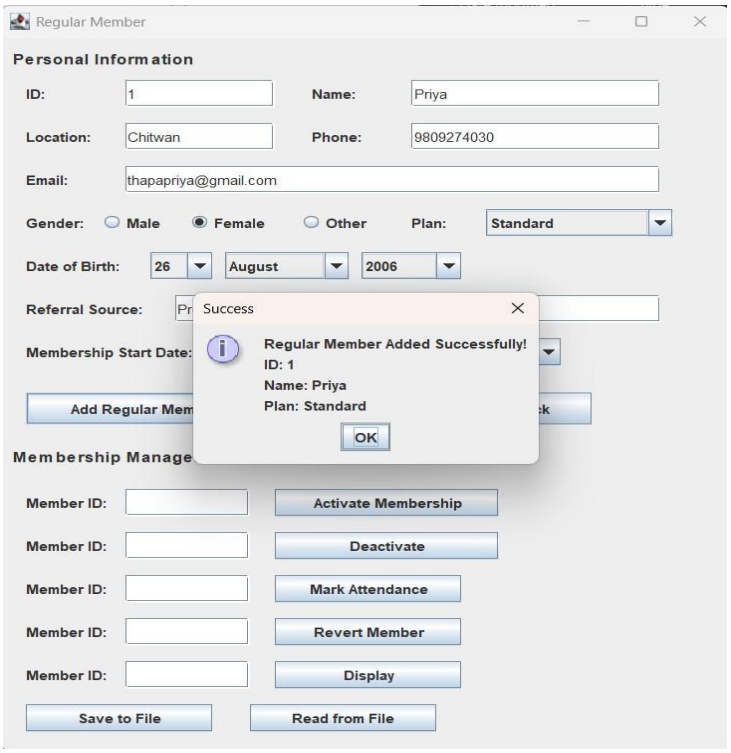

Figure 9 Regular member addbtn

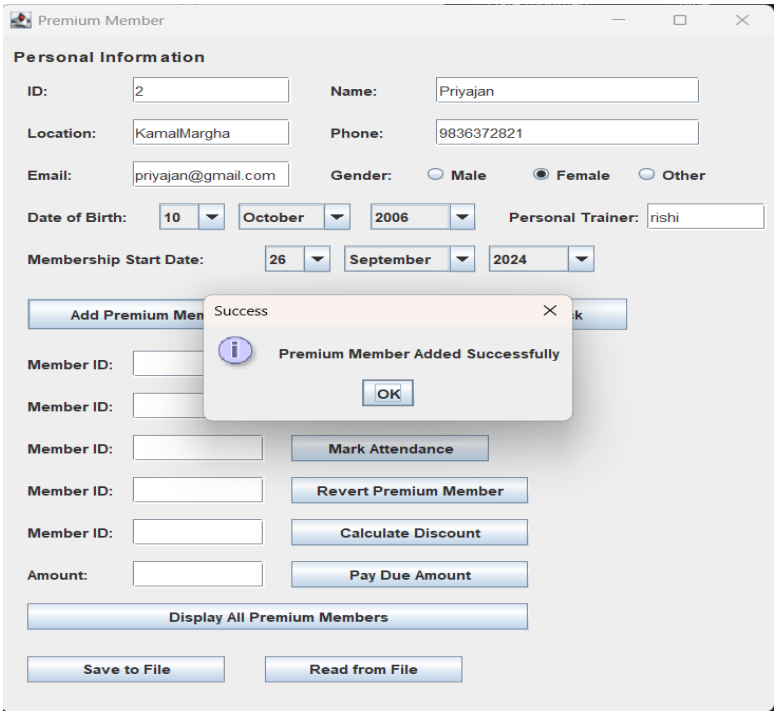

Figure 10 Premium member addbtn

Objective	To confirm that both regular and premium members are added properly by the software.
Action	1. Fill in all required fields for a regular and premium member. 2. Click “add Regular Member”. 3. Repeat for a premium member with relevant details.
Expected Output	Both members have been successfully added, and confirmation messages have been received.
Actual Output	The necessary confirmation messages have been delivered to both regular and premium members after clicking the add button.
Result	The test was successful

Table 2 Addbutton testing

6.3 Testing 3

6.3.1 Mark Attendance

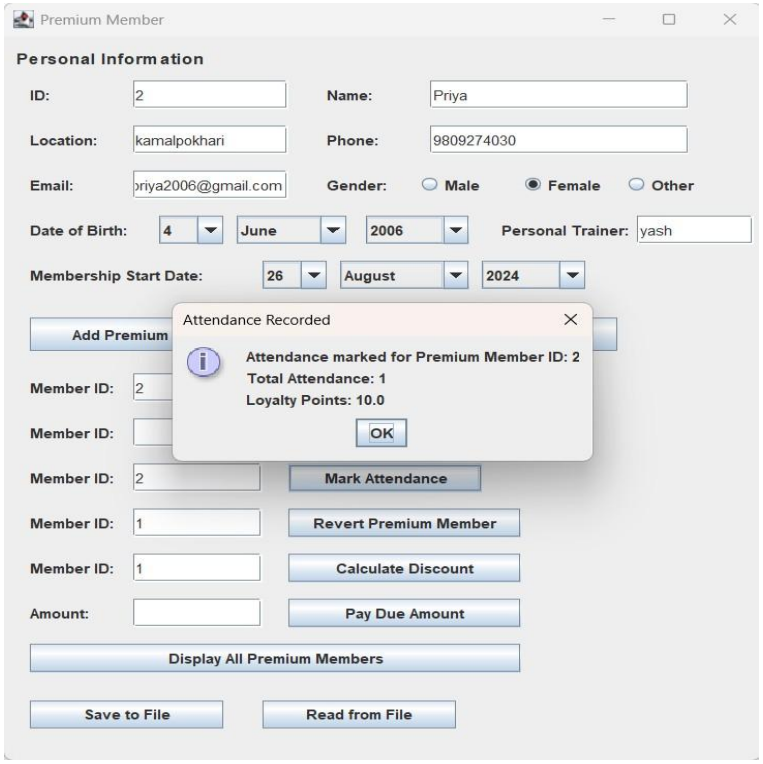

Figure 11 Test for Mark Attend

Objective	To verify attendance marking functionality works as expected.
Action	1. Fill all required personal information fields. 2. click the activate button. 3. click the "Mark Attendance "button.

Expected Output	After clicking the "Mark Attendance" button, the system should successfully record the attendance and automatically add the corresponding loyalty points to the respected regular and premium member.
Actual Output	After clicking the "Mark Attendance" button, the system successfully recorded the attendance and automatically added the loyalty points to the regular and premium member as expected.
Result	The test was successful

Table 3 Mark attendance button testing

6.3.2 Upgrade plan

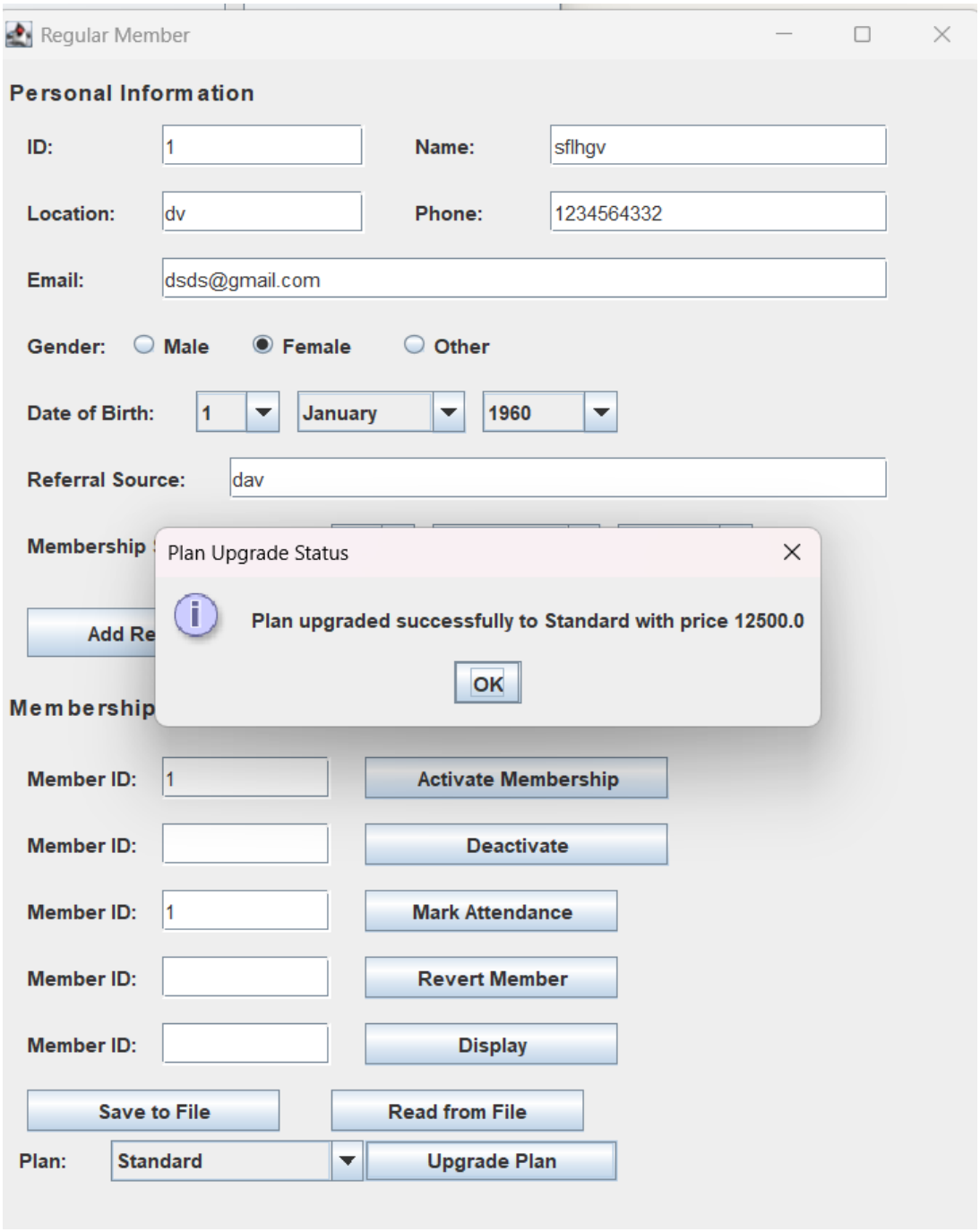

Figure 12 Upgrade testing

Objective	To verify upgrade plan functionality works as expected.
Action	Fill the information than mark the attendance when attendance reach more than 30 or equals to 30 then click the upgrade button.
Expected Output	After clicking the upgrade plan and change plan it should display a upgrade plan with price.
Actual Output	As expected the upgrade plan was work perfectly and also display the plan with price
Result	The test was successful

Table 4 Upgrade button testing

6.4 Testing 4

6.4.1 Pay due, calculate discount testing

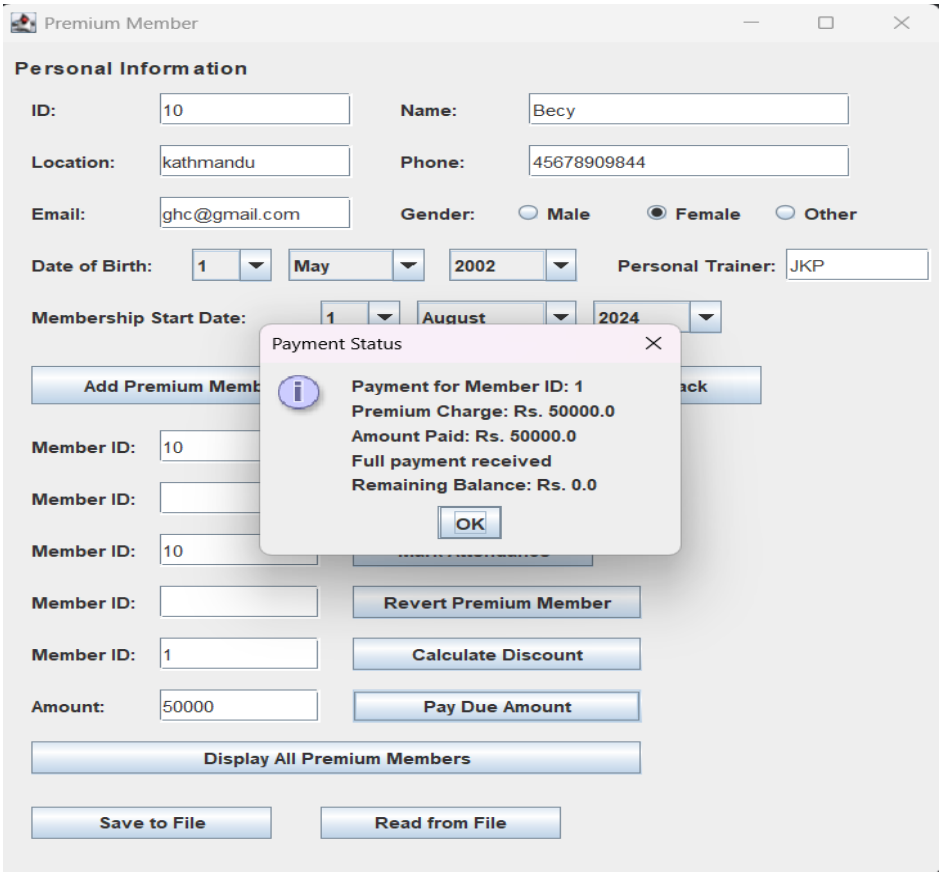

Figure 13 Pay due

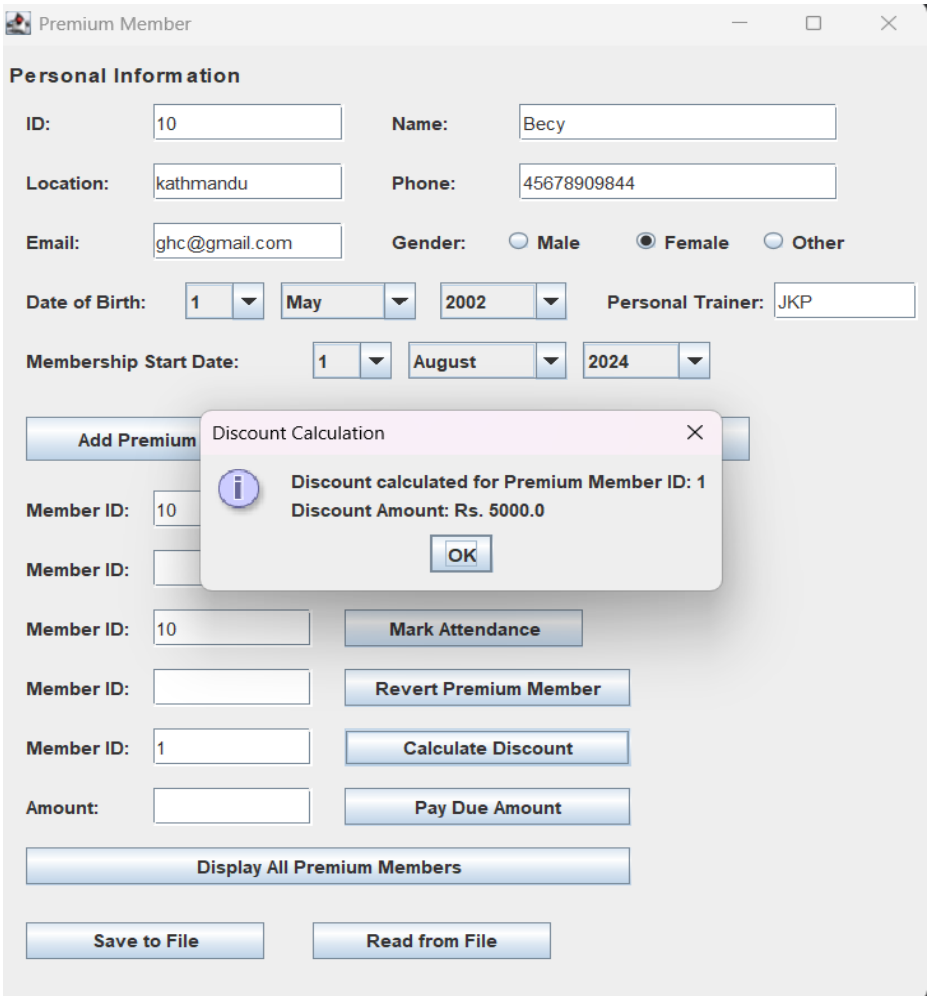

Figure 14 Calculate discount

Objective	To verify the pay due and calculate discount button
Action	Fill the information, add to regular member then active the account and click pay due button and then calculate button.
Expected Output	After adding payment click the pay due if pay amount is 50000 it will give discount if less than it will not give the discount
Actual Output	As expected, After adding payment click the pay due if pay amount is 50000 it did give discount and if less than it also didnt give the discount
Result	The test was successful

Table 5 Pay due and calculate discount testing

6.4.2 Revert Member testing

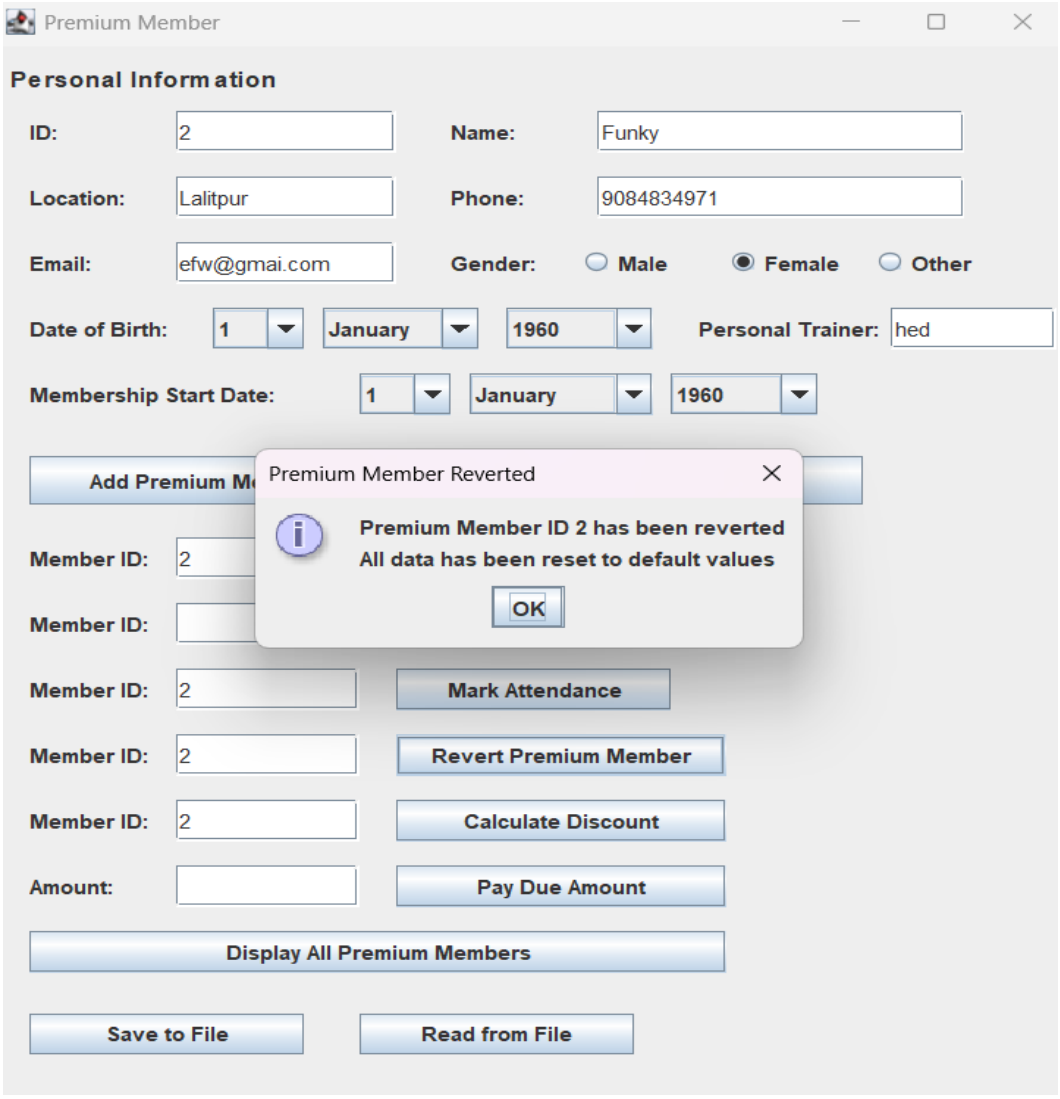

Figure 15 Revert Member

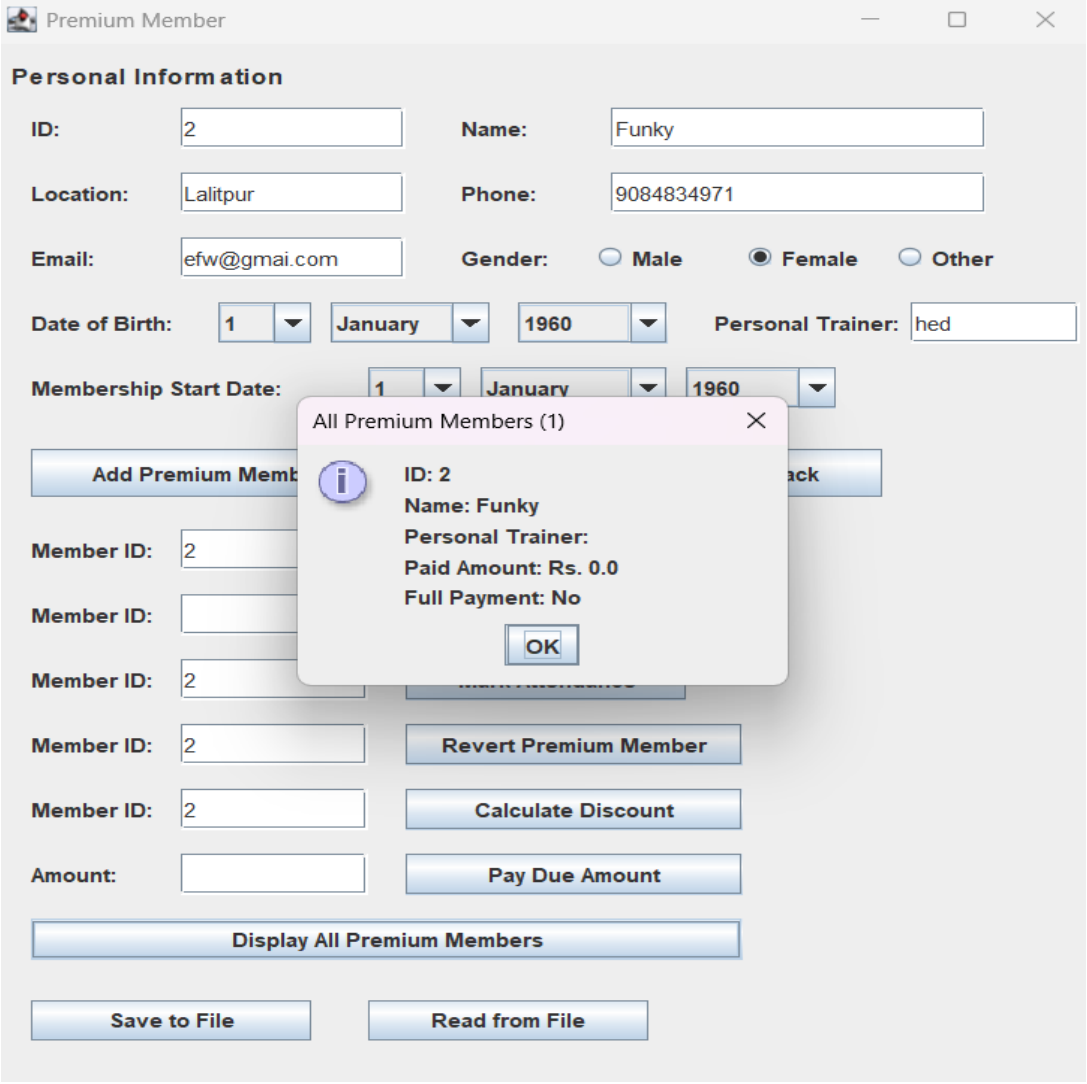

Figure 16 After clicking the button

Objective	To verify if the revert button works or not
Action	Fill the information, add to regular member then active the account and mark attendance than if user want to restart the plan than click the revert button.
Expected Output	When the revert button is clicked then it should reset the member plans, attendance, loyalpoint.
Actual Output	As expected, When the revert button was clicked then it does reset the member plans, attendance, loyalpoint.
Result	The test was successful

Table 6 Revert button testing

6..5 Testing 5 Save to and read from file

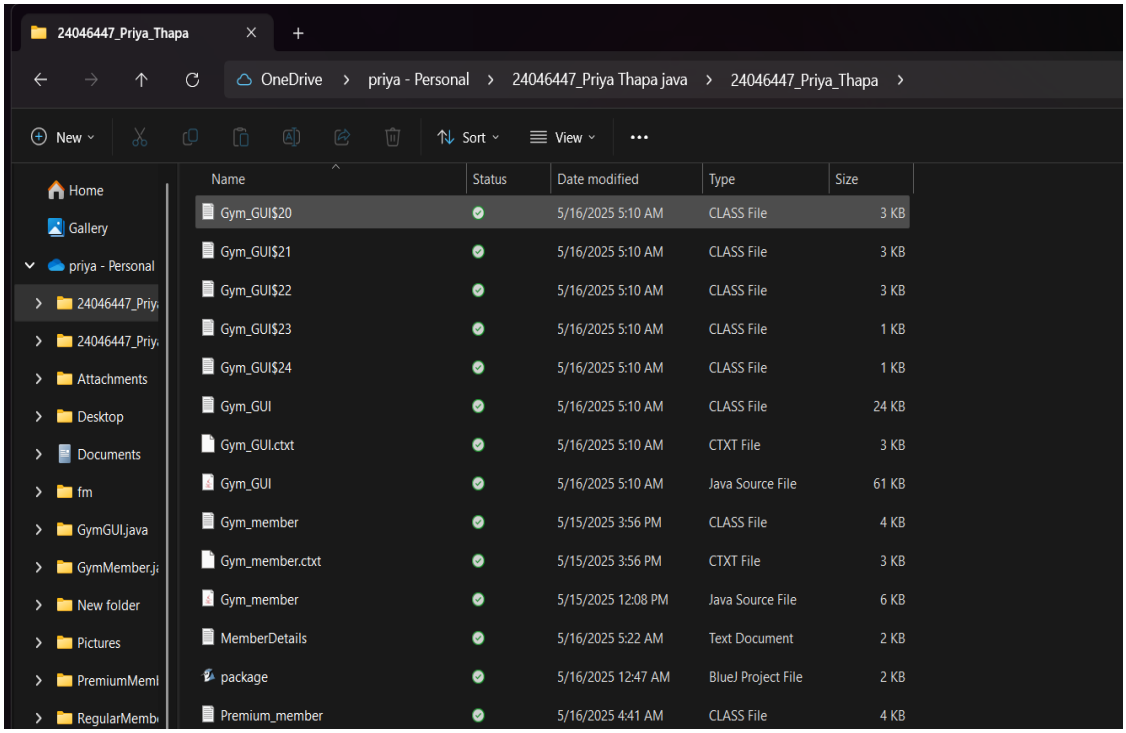

Figure 17 Save to file

Figure 18 Read to file

Objective	To verify the functionality of save to file and read to file
Action	Fill the all the information and then click to save to file foe save file and for read after clicking save to file click read to file.
Expected Output	When the save to file is clicked it will save the file name “MemberDetails.txt” in our file where are all codes are and in load to file it will read from the save to file.
Actual Output	As expected, When the save to file is clicked it does save the file in our file where are all codes are located in the form of txt file and after clicking read to file it actually does read from save file.
Result	The test was successful

Table 7 Testing for read and save to file

7.Error Detection and error

7.1 Syntax Error

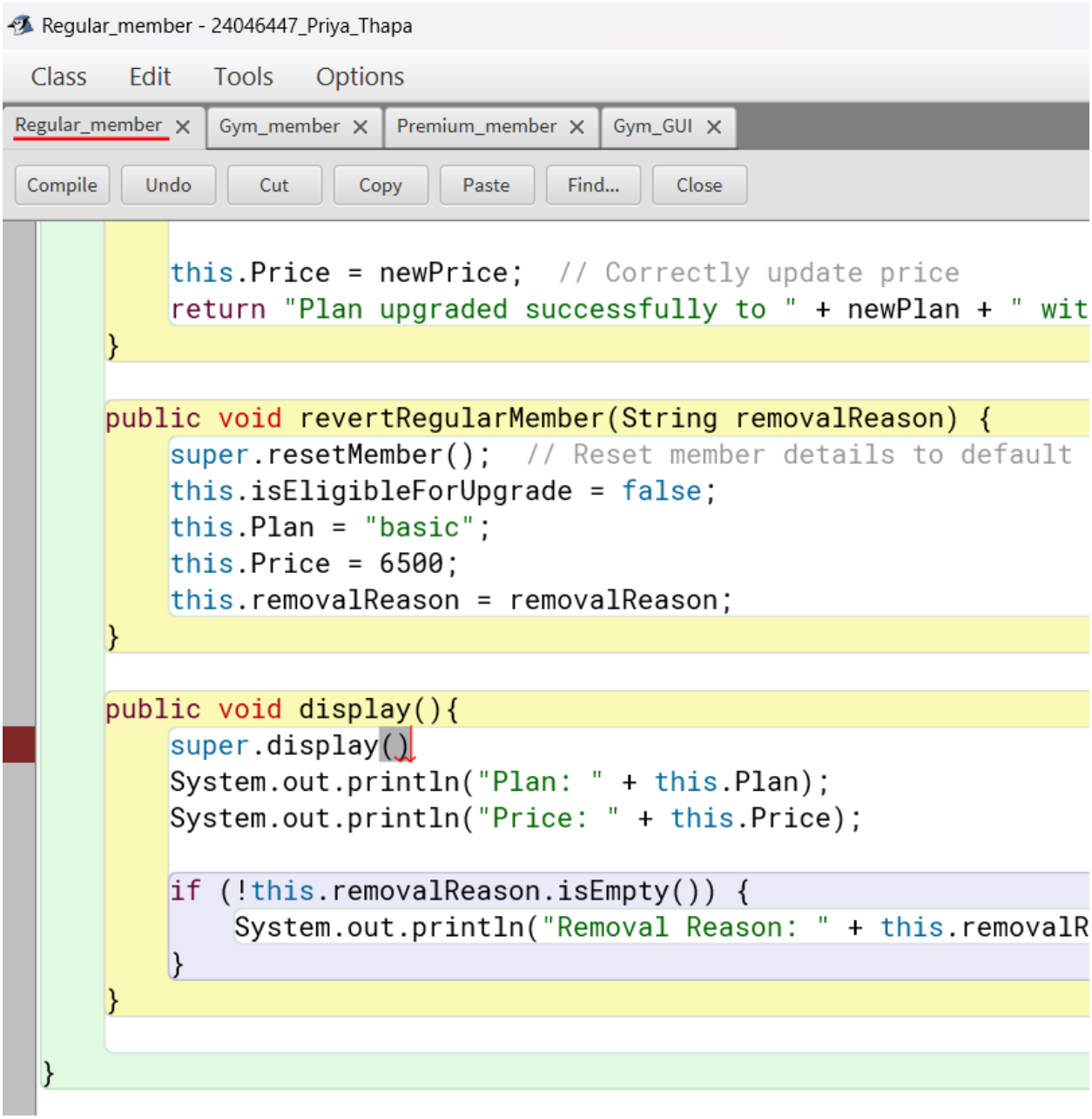

Figure 19 syntax error

In my above picture there was a syntax error where I forgot to write semicolon at the end of super.display() so I correct my programming by adding those semicolon now my program runs smoothly.

```java
/* Method to get the price based on the selected plan*/
public double getPlanPrice(String Plan){
    switch(Plan.toUpperCase()){
        case "basic":
            System.out.println("Price of this"+this.Plan+"plan is 6500");
            this.Price = 6500.0;
            return this.Price;

        case "standard":
            System.out.println("Price of this"+this.Plan+"plan is 12500");
            this.Price = 12500.0;
            return this.Price;

        case "deluxe":
            System.out.println("Price of this"+this.Plan+"plan is 18500");
            this.Price = 18500.0;
            return this.Price;

        default:
            return -1;

    }
}
```

Figure 20 syntax correction

7.2 Logical Error

```
/* Method to get the price based on the selected plan*/
public double getPlanPrice(String Plan){
    switch(Plan.toUpperCase()){
        case "basic":
            System.out.println("Price of this"+this.Plan+"plan is 6500");
            this.Price = 6500.0;
            return this.Price;

        case "standard":
            System.out.println("Price of this"+this.Plan+"plan is 12500");
            this.Price = 12500.0;
            return this.Price;

        case "deluxe":
            System.out.println("Price of this"+this.Plan+"plan is 18500");
            this.Price = 18500.0;
            return this.Price;

        default:
            return -1;

    }
}
```

Figure 21 Logical error in code

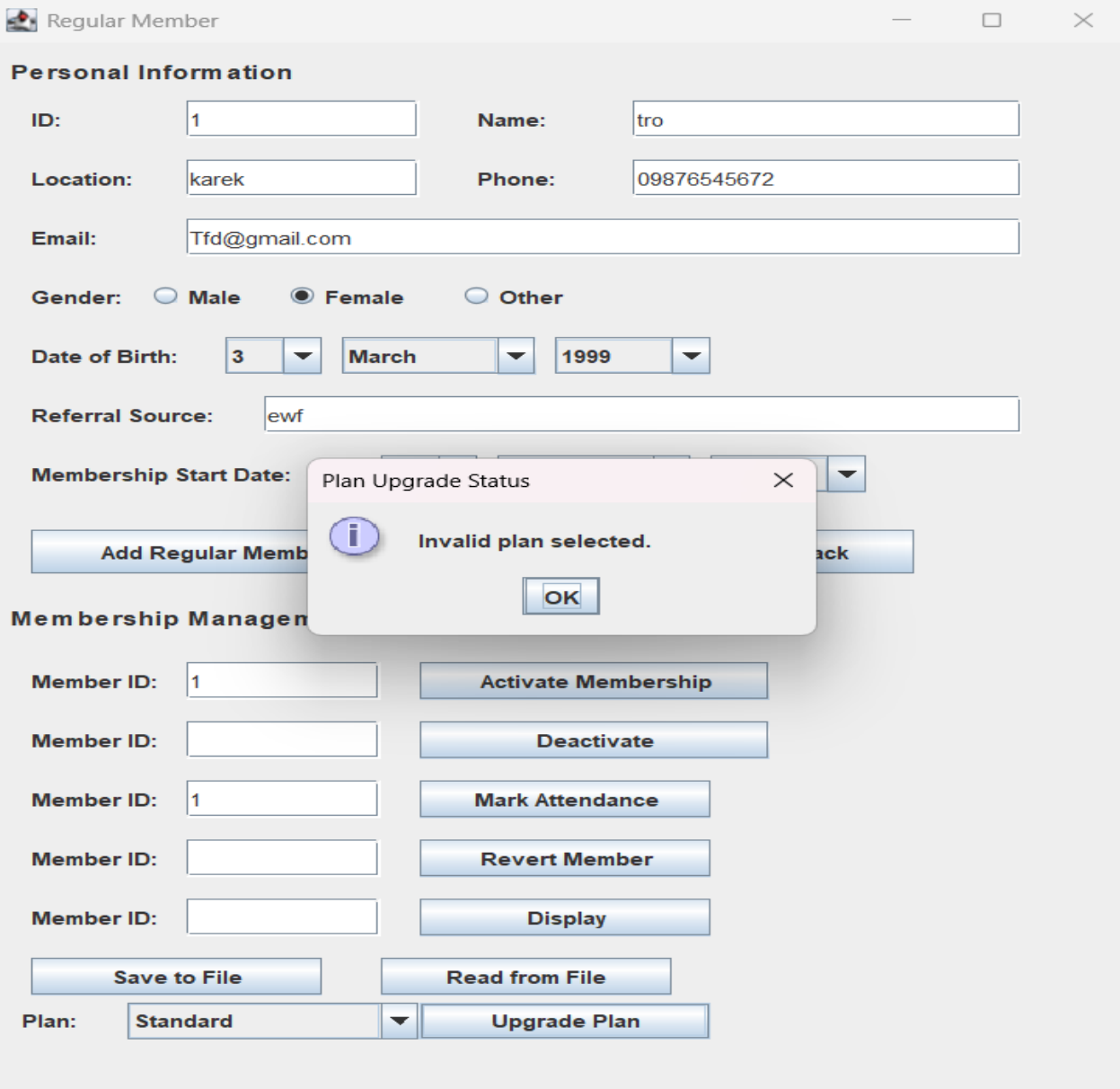

Figure 22 Logical error

As you can see there is an error I can't upgrade my plan because there is a logical error. In my code I add upper case function in my plan but in case I write in lower case which cause the logical error. Now I fix it into lower case as it run better than previous and there is no error it runs without any error.

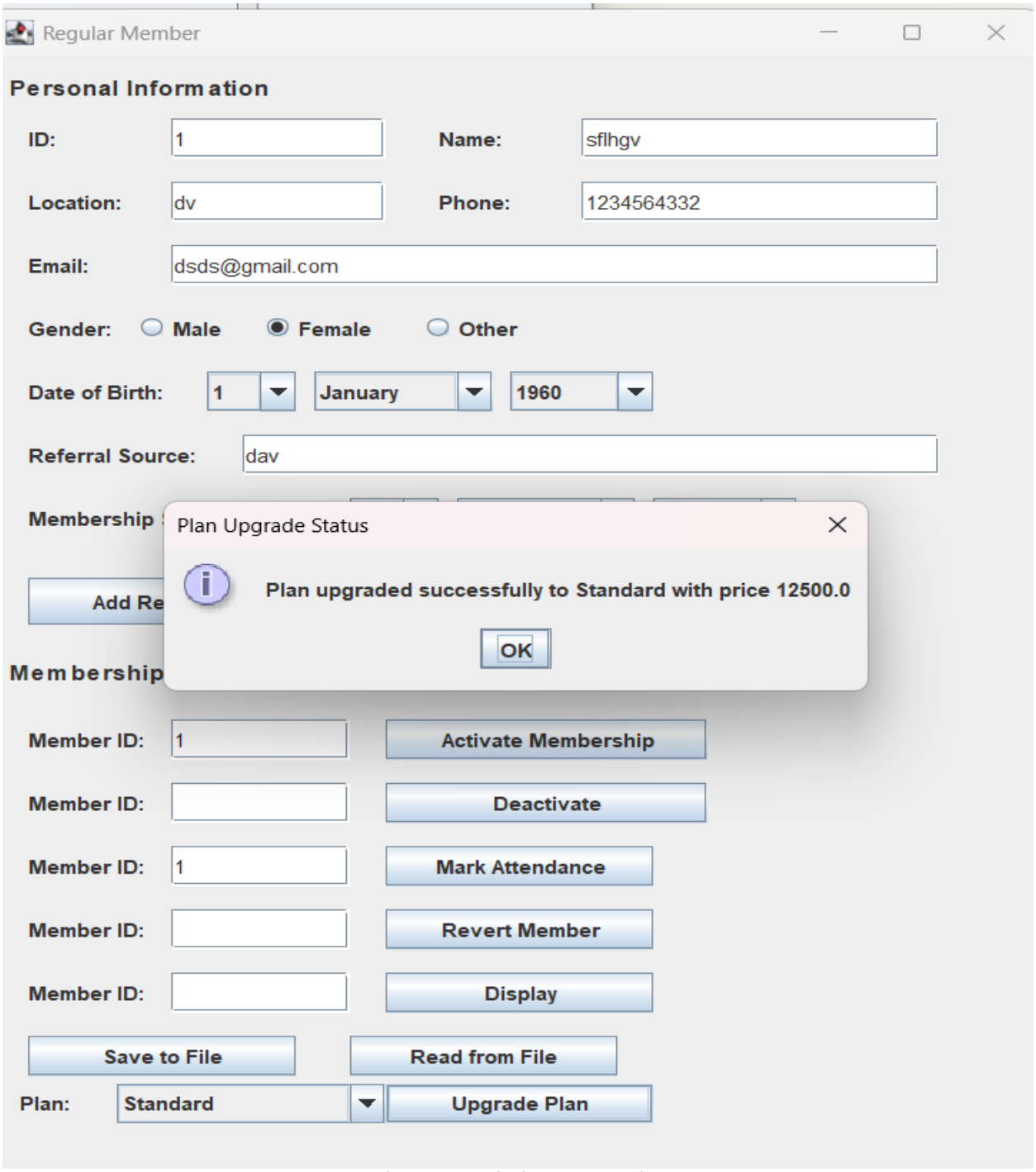

Figure 23 Logical error correction

7.3 Runtime Error

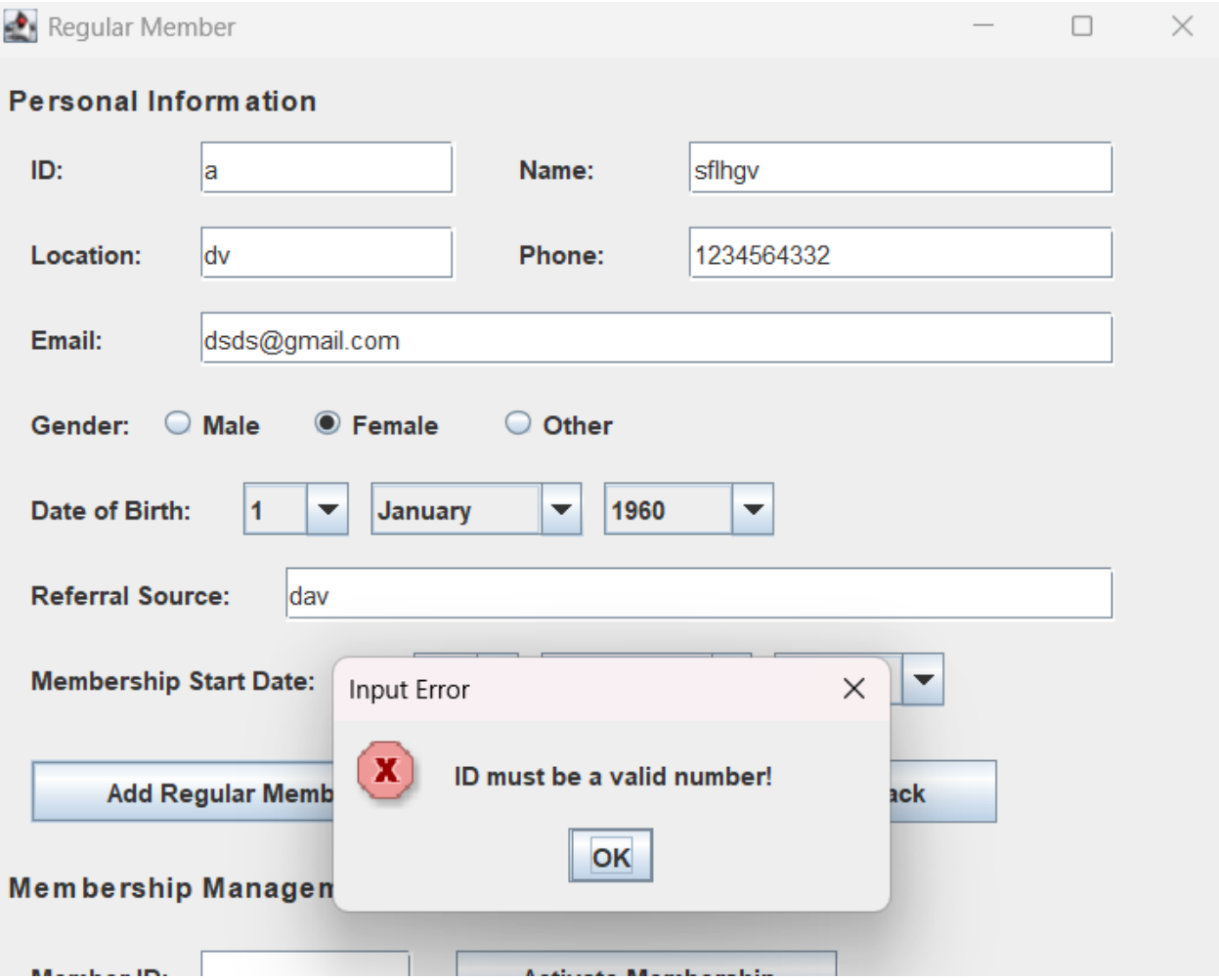

Figure 24 Runtime error

In this there is runtime error, when I write in non-numeric things in int it shows me the runtime error so I fixed it by putting number which gave me the output. At the end it shows runtime error in this because id can only take numeric only. So I fix all of it.

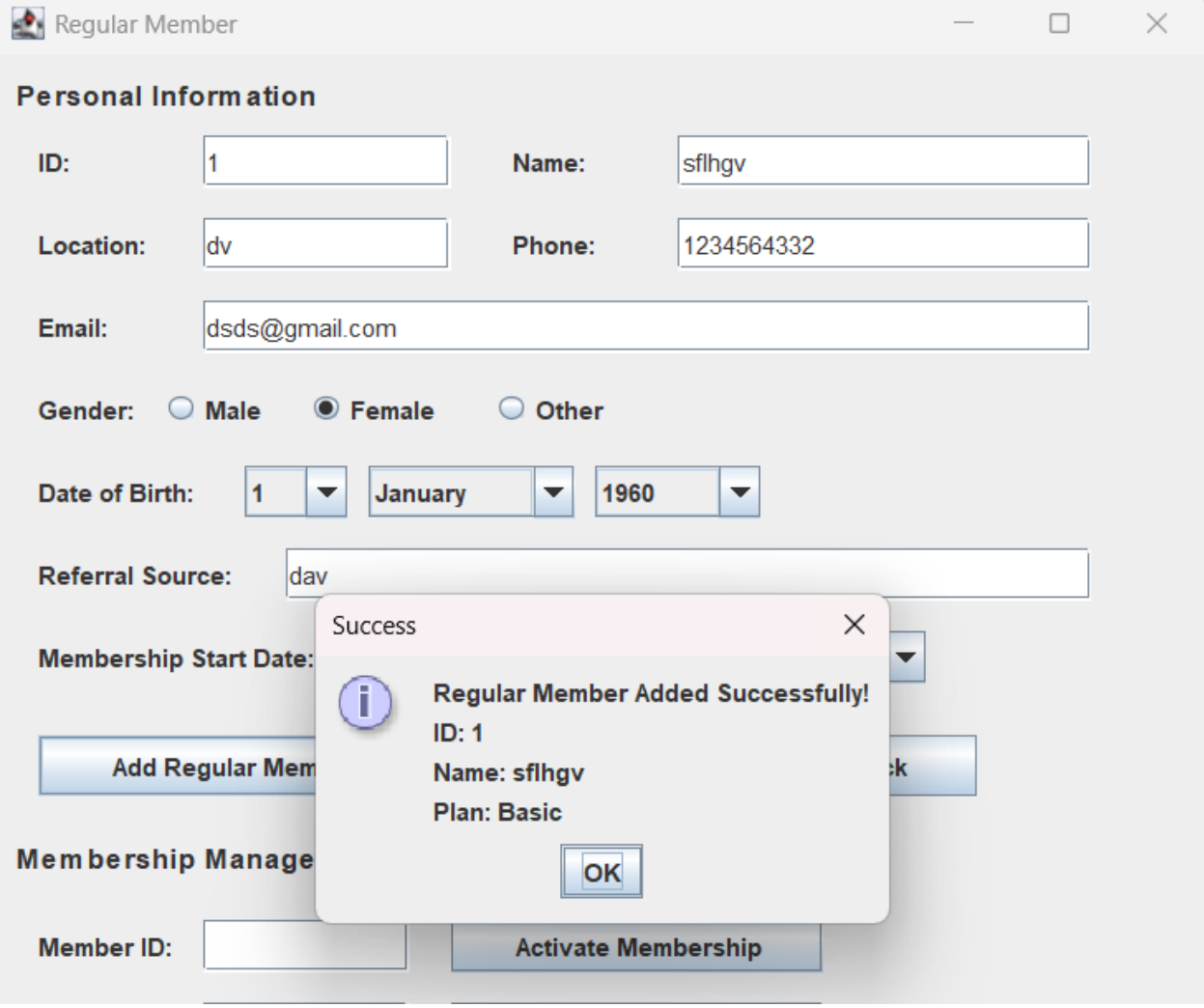

Figure 25 Runtime error correction

CONCLUSION

This project of the Gym Management System was successfully able to meet its objectives using important features such as member registration, attendance, payment, and file I/O-based data storage. Through the use of Java Swing and object-oriented programming principles, the system demonstrates the correct use of inheritance (for example, Premium_member is a subclass of Gym_member) and encapsulation.

The significant problems were resolving GUI layout problems, handling runtime exceptions (e.g., NullPointerException), and enhancing business logic for payment calculations. They were addressed systematically through iterative testing, validation checks, and peer code review. The project demonstrated the importance of modularity—such as improvements in the future can involve using a database (e.g., MySQL) for scalability purposes and utilizing JavaFX for the latest UI.

Beyond technical accomplishments, this coursework helped enhance my approach to problem-solving, consideration for edge cases, and ability to translate requirements to maintainable code. The process highlighted how theory OOP practice applies to the real world and prepared me to tackle more complex software engineering.

References

GeeksforGeeks. (2021, May 21). *Pseudocode*. Retrieved from GeeksforGeeks.: https://www.geeksforgeeks.org/what-is-pseudocode-a-complete-tutorial/

Interaction Design Foundation. (2024, May 10). *What is wireframe?* Retrieved from Interaction Design Foundation: https://l.instagram.com/?u=https%3A%2F%2Fwww.interaction-design.org%2Fliterature%2Ftopics%2Fwireframe&e=AT2Ck2-zBlBIKyuvE_Zfe_PGnKTPHrTg0AiEwaCAfwNt6zHwmI2dVfadz2GGM__C0ADlBWSmdd8V-Fo0smf6LgltOR5MsOmS96lZR9KAH-eM1eV3aAelOXxjyCWl6H25jvoZ7JB1PyO-

Lucichart. (2024, April 12). *What is class Diagram in UML?* Retrieved from Lucichart: https://l.instagram.com/?u=https%3A%2F%2Fwww.visual-paradigm.com%2Fguide%2Fuml-unified-modeling-language%2Fwhat-is-class-diagram%2F&e=AT3wKdHUecXXI2X8lZNKgQipTodPNRNUlBqRJf1pBodFjTpReacdaBIpkkGiCpEsYnO3XPhsvGr6FzO1PFkWBfkrssUI1HSU_Mml6GUB8_H2r8IP9BMx3aUJm

APPENDIX

Gym_GUI

```
/**
 *The Gym GUI is a Java Swing application for managing gym members. It allows adding regular and premium members, tracking payments, applying discounts, and viewing member details.
 *Features include membership activation/deactivation, payment processing, and a display table. The interface is user-friendly with organized input fields and buttons for easy navigation.
 * @author (Priya_Thapa)
 * @version (1/03/2025)
 */
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import java.io.FileWriter;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.File;

public class Gym_GUI {
    protected ArrayList<Gym_member> arr = new ArrayList<Gym_member>();
    protected JFrame GymFrame, RegularFrame, PremiumFrame;
    protected JPanel GymPanel, RegularPanel, PremiumPanel;
    protected JTextField ID,ID1,Name ,Name1, Location ,Location1,Phone1, Phone,
Email,Email1,
    ReferralSource,markAttendanceIdField,markAttendanceIdField1
,displayMemberIdField,displayMemberIdField1,revertMemberIdField,PersonalTrainer,re
```

```
vertPremiumIdField,deactivateIdField,
    activateIdField,deactivateIdField1,activatePremiumIdField;
    protected JButton addRegular, addPremium,addPremium2, addRegular2,
activateBtn,

deactivateBtn,regularClearButton,backregularButton,DisplayButton,backPremiumButton
,DisplayButton1,activateBtn1,markAttendanceBtn1,revertPremiumBtn,
    deactivateBtn1,PremiumClearButton;
    protected JRadioButton radioButton1, radioButton2, radioButton3,radioButtonn1,
radioButtonn2, radioButtonn3;
    protected JComboBox<Integer> dayBox, yearBox, msdDayBox,
msdYearBox,dayBox1, yearBox1, msdDayBox1, msdYearBox1;
    protected JComboBox<String> monthBox, msdMonthBox,planBox,monthBox1,
msdMonthBox1;

    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    Gym_GUI window = new Gym_GUI();
                    window.GymFrame.setVisible(true);
                } catch (Exception e) {e.printStackTrace();
                }
            }
        });
    }

    public Gym_GUI() {
        arr = new ArrayList<>();

        // Main Frame
```

```
GymFrame = new JFrame("Gym Management System");
GymFrame.setBounds(100, 100, 400, 200);
GymFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

GymPanel = new JPanel();
GymPanel.setLayout(new GridLayout(1, 2, 10, 10));

addRegular = new JButton("Add Regular Member");
addRegular.addActionListener(new RegularButtonListener(this));

addPremium = new JButton("Add Premium Member");
addPremium.addActionListener(new PremiumButtonListener(this));

GymPanel.add(addRegular);
GymPanel.add(addPremium);
GymFrame.getContentPane().add(GymPanel, BorderLayout.CENTER);

// Regular Member Frame
RegularFrame = new JFrame("Regular Member");
RegularFrame.setBounds(150, 170, 600, 750);
RegularFrame.setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);

RegularPanel = new JPanel();
RegularPanel.setLayout(null);
RegularPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

// Personal Information Section
JLabel personalInfoLabel = new JLabel("Personal Information");
personalInfoLabel.setBounds(10, 10, 200, 20);
personalInfoLabel.setFont(new Font("Arial", Font.BOLD, 14));
RegularPanel.add(personalInfoLabel);
```

```
// ID
JLabel idLabel = new JLabel("ID:");
idLabel.setBounds(20, 40, 80, 25);
RegularPanel.add(idLabel);
ID = new JTextField();
ID.setBounds(100, 40, 120, 25);
RegularPanel.add(ID);

// Name
JLabel nameLabel = new JLabel("Name:");
nameLabel.setBounds(250, 40, 80, 25);
RegularPanel.add(nameLabel);
Name = new JTextField();
Name.setBounds(330, 40, 200, 25);
RegularPanel.add(Name);

// Location
JLabel locationLabel = new JLabel("Location:");
locationLabel.setBounds(20, 80, 80, 25);
RegularPanel.add(locationLabel);
Location = new JTextField();
Location.setBounds(100, 80, 120, 25);
RegularPanel.add(Location);

// Phone
JLabel phoneLabel = new JLabel("Phone:");
phoneLabel.setBounds(250, 80, 80, 25);
RegularPanel.add(phoneLabel);
Phone = new JTextField();
Phone.setBounds(330, 80, 200, 25);
```

```
RegularPanel.add(Phone);

// Email
JLabel emailLabel = new JLabel("Email:");
emailLabel.setBounds(20, 120, 80, 25);
RegularPanel.add(emailLabel);
Email = new JTextField();
Email.setBounds(100, 120, 430, 25);
RegularPanel.add(Email);

// Gender
JLabel genderLabel = new JLabel("Gender:");
genderLabel.setBounds(20, 160, 80, 25);
RegularPanel.add(genderLabel);

radioButton1 = new JRadioButton("Male");
radioButton1.setBounds(80, 160, 60, 25);
radioButton2 = new JRadioButton("Female");
radioButton2.setBounds(150, 160, 80, 25);
radioButton3 = new JRadioButton("Other");
radioButton3.setBounds(240, 160, 80, 25);

// Plan Selection for Regular Members
JLabel planLabel = new JLabel("Plan:");
planLabel.setBounds(15, 650, 80, 25);
RegularPanel.add(planLabel);

String[] plans = {"Basic", "Standard", "Deluxe"};
planBox = new JComboBox<>(plans);
planBox.setBounds(70, 650, 150, 25);
RegularPanel.add(planBox);
```

```
ButtonGroup genderGroup = new ButtonGroup();
genderGroup.add(radioButton1);
genderGroup.add(radioButton2);
genderGroup.add(radioButton3);

RegularPanel.add(radioButton1);
RegularPanel.add(radioButton2);
RegularPanel.add(radioButton3);

// DOB
JLabel dobLabel = new JLabel("Date of Birth:");
dobLabel.setBounds(20, 200, 100, 25);
RegularPanel.add(dobLabel);

Integer[] days = new Integer[31];
for (int i = 0; i < 31; i++)
    days[i] = i + 1;
dayBox = new JComboBox<>(days);
dayBox.setBounds(120, 200, 50, 25);
RegularPanel.add(dayBox);

String[] months = {
        "January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"
    };
monthBox = new JComboBox<>(months);
monthBox.setBounds(180, 200, 100, 25);
RegularPanel.add(monthBox);

int currentYear = java.time.Year.now().getValue();
```

```
Integer[] years = new Integer[currentYear - 1960 + 1];
for (int i = 0; i < years.length; i++)
    years[i] = 1960 + i;
yearBox = new JComboBox<>(years);
yearBox.setBounds(290, 200, 80, 25);
RegularPanel.add(yearBox);

// Referral Source
JLabel referralLabel = new JLabel("Referral Source:");
referralLabel.setBounds(20, 240, 120, 25);
RegularPanel.add(referralLabel);
ReferralSource = new JTextField();
ReferralSource.setBounds(140, 240, 390, 25);
RegularPanel.add(ReferralSource);

// Membership Start Date
JLabel msdLabel = new JLabel("Membership Start Date:");
msdLabel.setBounds(20, 280, 180, 25);
RegularPanel.add(msdLabel);

msdDayBox = new JComboBox<>(days);
msdDayBox.setBounds(200, 280, 50, 25);
RegularPanel.add(msdDayBox);

msdMonthBox = new JComboBox<>(months);
msdMonthBox.setBounds(260, 280, 100, 25);
RegularPanel.add(msdMonthBox);

msdYearBox = new JComboBox<>(years);
msdYearBox.setBounds(370, 280, 80, 25);
RegularPanel.add(msdYearBox);
```

```
// Add Member Button
addRegular2 = new JButton("Add Regular Member");
addRegular2.setBounds(20, 330, 190, 30);
addRegular2.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            addRegularMember();
        }
    });
RegularPanel.add(addRegular2);

// Membership Management Section
JLabel membershipLabel = new JLabel("Membership Management");
membershipLabel.setBounds(10, 380, 200, 20);
membershipLabel.setFont(new Font("Arial", Font.BOLD, 14));
RegularPanel.add(membershipLabel);

// Activate Membership
JLabel activateLabel = new JLabel("Member ID:");
activateLabel.setBounds(20, 420, 80, 25);
RegularPanel.add(activateLabel);

activateIdField = new JTextField();
activateIdField.setBounds(100, 420, 100, 25);
RegularPanel.add(activateIdField);

activateBtn = new JButton("Activate Membership");
activateBtn.setBounds(220, 420, 180, 25);
activateBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            String inputId = activateIdField.getText().trim();
```

```
                try {
                    int memberId = Integer.parseInt(inputId);
                    Gym_member member = findMemberById(memberId);
                    if (member != null) {
                        member.activateMembership();
                        JOptionPane.showMessageDialog(RegularFrame, "Membership
Activated for Member ID: " + memberId);
                    } else {
                        JOptionPane.showMessageDialog(RegularFrame, "Member ID not
found.");
                    }
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(RegularFrame, "Invalid Member ID
entered.");
                }
            }
        });
    RegularPanel.add(activateBtn);

    // Deactivate Membership
    JLabel deactivateLabel = new JLabel("Member ID:");
    deactivateLabel.setBounds(20, 460, 80, 25);
    RegularPanel.add(deactivateLabel);

    deactivateIdField = new JTextField();
    deactivateIdField.setBounds(100, 460, 100, 25);
    RegularPanel.add(deactivateIdField);

    deactivateBtn = new JButton("Deactivate");
    deactivateBtn.setBounds(220, 460, 180, 25);
    deactivateBtn.addActionListener(new ActionListener() {
```

```
            public void actionPerformed(ActionEvent e) {
                String inputId = deactivateIdField.getText().trim();
                try {
                    int memberId = Integer.parseInt(inputId);
                    Gym_member member = findMemberById(memberId);
                    if (member != null) {
                        member.deactiveMembership();
                        JOptionPane.showMessageDialog(RegularFrame, "Membership
Deactivated for Member ID: " + memberId);
                    } else {
                        JOptionPane.showMessageDialog(RegularFrame, "Member ID not
found.");
                    }
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(RegularFrame, "Invalid Member ID
entered.");
                }
            }
        });
    RegularPanel.add(deactivateBtn);

    JLabel markAttendanceLabel = new JLabel("Member ID:");
    markAttendanceLabel.setBounds(20, 500, 80, 25);
    RegularPanel.add(markAttendanceLabel);

    markAttendanceIdField = new JTextField();
    markAttendanceIdField.setBounds(100, 500, 100, 25);
    RegularPanel.add(markAttendanceIdField);

    JButton markAttendanceBtn = new JButton("Mark Attendance");
    markAttendanceBtn.setBounds(220, 500, 150, 25);
```

```
markAttendanceBtn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String inputId = markAttendanceIdField.getText().trim();
        try {
            int memberId = Integer.parseInt(inputId);
            Gym_member member = findMemberById(memberId);

            if (member != null && member instanceof Regular_member) {
                if (member.get_ActiveStatus()) {
                    member.markAttendance();
                    JOptionPane.showMessageDialog(RegularFrame,
                        "Attendance marked for Regular Member ID: " + memberId +
                        "\nTotal Attendance: " + member.get_Attendance() +
                        "\nLoyalty Points: " + member.get_LoyalPoints(),
                        "Attendance Recorded",
                        JOptionPane.INFORMATION_MESSAGE);
                } else {
                    JOptionPane.showMessageDialog(RegularFrame,
                        "Cannot mark attendance - membership is inactive",
                        "Inactive Membership",
                        JOptionPane.WARNING_MESSAGE);
                }
            } else {
                JOptionPane.showMessageDialog(RegularFrame,
                    "Regular Member ID not found",
                    "Not Found",
                    JOptionPane.WARNING_MESSAGE);
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(RegularFrame,
                "Please enter a valid numeric Member ID",
```

```
                    "Invalid Input",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    });
RegularPanel.add(markAttendanceBtn);

JLabel revertMemberLabel = new JLabel("Member ID:");
revertMemberLabel.setBounds(20, 540, 80, 25);
RegularPanel.add(revertMemberLabel);

revertMemberIdField = new JTextField();
revertMemberIdField.setBounds(100, 540, 100, 25);
RegularPanel.add(revertMemberIdField);

JButton revertMemberBtn = new JButton("Revert Member");
revertMemberBtn.setBounds(220, 540, 150, 25);
revertMemberBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            String inputId = revertMemberIdField.getText().trim();
            try {
                int memberId = Integer.parseInt(inputId);
                Gym_member member = findMemberById(memberId);

                if (member != null && member instanceof Regular_member) {
                    ((Regular_member)member).revertRegularMember("Admin request");
                    JOptionPane.showMessageDialog(RegularFrame,
                        "Regular Member ID " + memberId + " has been reverted\n" +
                        "All data has been reset to default values",
                        "Member Reverted",
                        JOptionPane.INFORMATION_MESSAGE);
```

```
                } else {
                    JOptionPane.showMessageDialog(RegularFrame,
                        "Regular Member ID not found",
                        "Not Found",
                        JOptionPane.WARNING_MESSAGE);
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(RegularFrame,
                    "Please enter a valid numeric Member ID",
                    "Invalid Input",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    });
RegularPanel.add(revertMemberBtn);

// Clear Button
regularClearButton = new JButton("Clear");
regularClearButton.setBounds(270, 330, 95, 30);
regularClearButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            ID.setText("");
            Name.setText("");
            Location.setText("");
            Phone.setText("");
            Email.setText("");
            ReferralSource.setText("");
            planBox.setSelectedIndex(-1);

            radioButton1.setSelected(false);
```

```
            radioButton2.setSelected(false);
            radioButton3.setSelected(false);

            dayBox.setSelectedIndex(0);
            monthBox.setSelectedIndex(0);
            yearBox.setSelectedIndex(0);
            msdDayBox.setSelectedIndex(0);
            msdMonthBox.setSelectedIndex(0);
            msdYearBox.setSelectedIndex(0);
        }
    });
RegularPanel.add(regularClearButton);

// Back Button
backregularButton = new JButton("Back");
backregularButton.setBounds(380, 330, 95, 30);
backregularButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            RegularFrame.setVisible(false);
            GymFrame.setVisible(true);
        }
    });
RegularPanel.add(backregularButton);

JLabel displayMemberLabel = new JLabel("Member ID:");
displayMemberLabel.setBounds(20, 580, 80, 25);
RegularPanel.add(displayMemberLabel);

final JTextField displayMemberIdField = new JTextField();
displayMemberIdField.setBounds(100, 580, 100, 25);
```

```
        RegularPanel.add(displayMemberIdField);

        DisplayButton = new JButton("Display");
        DisplayButton.setBounds(220, 580, 150, 25);
        DisplayButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                String inputId = displayMemberIdField.getText().trim();
                try {
                    int memberId = Integer.parseInt(inputId);
                    Gym_member member = findMemberById(memberId);

                    if (member != null && member instanceof Regular_member) {
                        Regular_member regularMember = (Regular_member) member;
                        String info = "ID: " + regularMember.get_id() + "\n" +
                            "Name: " + regularMember.get_name() + "\n" +
                            "Location: " + regularMember.get_Location() + "\n" +
                            "Phone: " + regularMember.get_Phone()+ "\n" +
                            "Email: " + regularMember.get_Gender() + "\n" +
                            "Referral Source: " + regularMember.get_referralSource() + "\n" +
                            "Gender: " + regularMember.get_Gender() + "\n" +
                            "Date of Birth: " + regularMember.get_DOB() + "\n" +
                            "Membership Start Date: " +
regularMember.get_MembershipStartDate() + "\n" +
                            "Active Status: " + regularMember.get_ActiveStatus() + "\n" +
                            "Attendance: " + regularMember.get_Attendance() + "\n" +
                            "Loyalty Points: " + regularMember.get_LoyalPoints() + "\n" +
                            "Plan: " + regularMember.get_Plan();

                        JOptionPane.showMessageDialog(RegularFrame, info, "Regular
Member Information", JOptionPane.INFORMATION_MESSAGE);
```

```
                } else if (member != null) {
                    JOptionPane.showMessageDialog(RegularFrame, "Member with ID "
+ memberId + " is not a Regular Member.", "Not a Regular Member",
JOptionPane.WARNING_MESSAGE);
                } else {
                    JOptionPane.showMessageDialog(RegularFrame, "Member with ID "
+ memberId + " not found.", "Not Found", JOptionPane.WARNING_MESSAGE);
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(RegularFrame, "Invalid Member ID
format. Please enter a number.", "Invalid Input", JOptionPane.ERROR_MESSAGE);
            }
        }
    });
    RegularPanel.add(DisplayButton);

    JButton upgradeBtn = new JButton("Upgrade Plan");
    upgradeBtn.setBounds(220, 650, 150, 25);
    upgradeBtn.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            // Get the member ID from the ID field
            String memberIdStr = ID.getText().trim();

            if (memberIdStr.isEmpty()) {
                JOptionPane.showMessageDialog(RegularFrame,
                    "Please enter a Member ID first",
                    "Input Error",
                    JOptionPane.WARNING_MESSAGE);
                return;
            }
```

```
try {
    int memberId = Integer.parseInt(memberIdStr);
    String plan = (String) planBox.getSelectedItem();

    if (plan == null || plan.isEmpty()) {
        JOptionPane.showMessageDialog(RegularFrame,
            "Please select a plan to upgrade to",
            "Input Error",
            JOptionPane.WARNING_MESSAGE);
        return;
    }

    Gym_member member = findMemberById(memberId);

    if (member == null) {
        JOptionPane.showMessageDialog(RegularFrame,
            "Member with ID " + memberId + " not found",
            "Not Found",
            JOptionPane.WARNING_MESSAGE);
        return;
    }

    if (!(member instanceof Regular_member)) {
        JOptionPane.showMessageDialog(RegularFrame,
            "Member with ID " + memberId + " is not a Regular Member",
            "Wrong Member Type",
            JOptionPane.WARNING_MESSAGE);
        return;
    }
```

```
                if (!member.get_ActiveStatus()) {
                    JOptionPane.showMessageDialog(RegularFrame,
                        "Cannot upgrade plan - membership is inactive",
                        "Inactive Membership",
                        JOptionPane.WARNING_MESSAGE);
                    return;
                }

                Regular_member regularMember = (Regular_member) member;
                String result = regularMember.upgradePlan(plan);

                JOptionPane.showMessageDialog(RegularFrame,
                    result,
                    "Plan Upgrade Status",
                    JOptionPane.INFORMATION_MESSAGE);

            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(RegularFrame,
                    "Please enter a valid numeric Member ID",
                    "Invalid Input",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    });
RegularPanel.add(upgradeBtn);

// Add these to your RegularPanel setup (place them where you want)
JButton regularSaveBtn = new JButton("Save to File");
regularSaveBtn.setBounds(20, 620, 150, 25);
regularSaveBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
```

```
            save(arr);  // Call the save method with the member list
        }
    });

    JButton regularLoadBtn = new JButton("Read from File");
    regularLoadBtn.setBounds(200, 620, 150, 25);
    regularLoadBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            read();  // Call the read method
        }
    });

    // Add to RegularPanel
    RegularPanel.add(regularSaveBtn);
    RegularPanel.add(regularLoadBtn);

    RegularFrame.getContentPane().add(RegularPanel);

    // Premium Member Frame
    PremiumFrame = new JFrame("Premium Member");
    PremiumFrame.setBounds(150, 150, 600, 680);
    PremiumFrame.setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);

    PremiumPanel = new JPanel();
    PremiumPanel.setLayout(null);
    PremiumPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    // Personal Information Section
    JLabel personalInfoLabel1 = new JLabel("Personal Information");
    personalInfoLabel1.setBounds(10, 10, 200, 20);
    personalInfoLabel1.setFont(new Font("Arial", Font.BOLD, 14));
```

```
PremiumPanel.add(personalInfoLabel1);

// ID
JLabel idLabel1 = new JLabel("ID:");
idLabel1.setBounds(20, 40, 80, 25);
PremiumPanel.add(idLabel1);
ID1 = new JTextField();
ID1.setBounds(100, 40, 120, 25);
PremiumPanel.add(ID1);

// Name
JLabel nameLabel1 = new JLabel("Name:");
nameLabel1.setBounds(250, 40, 80, 25);
PremiumPanel.add(nameLabel1);
Name1 = new JTextField();
Name1.setBounds(330, 40, 200, 25);
PremiumPanel.add(Name1);

// Location
JLabel locationLabel1 = new JLabel("Location:");
locationLabel1.setBounds(20, 80, 80, 25);
PremiumPanel.add(locationLabel1);
Location1 = new JTextField();
Location1.setBounds(100, 80, 120, 25);
PremiumPanel.add(Location1);

// Phone
JLabel phoneLabel1 = new JLabel("Phone:");
phoneLabel1.setBounds(250, 80, 80, 25);
PremiumPanel.add(phoneLabel1);
Phone1 = new JTextField();
```

```
Phone1.setBounds(330, 80, 200, 25);
PremiumPanel.add(Phone1);

// Email
JLabel emailLabel1 = new JLabel("Email:");
emailLabel1.setBounds(20, 120, 80, 25);
PremiumPanel.add(emailLabel1);
Email1 = new JTextField();
Email1.setBounds(100, 120, 120, 25);
PremiumPanel.add(Email1);

// Gender
JLabel genderLabel1 = new JLabel("Gender:");
genderLabel1.setBounds(250, 120, 80, 25);
PremiumPanel.add(genderLabel1);

radioButtonn1 = new JRadioButton("Male");
radioButtonn1.setBounds(320, 120, 60, 25);
radioButtonn2 = new JRadioButton("Female");
radioButtonn2.setBounds(400, 120, 80, 25);
radioButtonn3 = new JRadioButton("Other");
radioButtonn3.setBounds(480, 120, 80, 25);

ButtonGroup genderGroup1 = new ButtonGroup();
genderGroup1.add(radioButtonn1);
genderGroup1.add(radioButtonn2);
genderGroup1.add(radioButtonn3);

PremiumPanel.add(radioButtonn1);
PremiumPanel.add(radioButtonn2);
PremiumPanel.add(radioButtonn3);
```

```
JLabel dobLabel1 = new JLabel("Date of Birth:");
dobLabel1.setBounds(20, 160, 100, 25);
PremiumPanel.add(dobLabel1);

Integer[] days1 = new Integer[31];
for (int i = 0; i < 31; i++)
    days[i] = i + 1;
dayBox1 = new JComboBox<>(days);
dayBox1.setBounds(120, 160, 50, 25);
PremiumPanel.add(dayBox1);

String[] months1 = {
        "January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"
    };
monthBox1 = new JComboBox<>(months1);
monthBox1.setBounds(180, 160, 85, 25);
PremiumPanel.add(monthBox1);

int currentYear1 = java.time.Year.now().getValue();
Integer[] years1 = new Integer[currentYear1 - 1960 + 1];
for (int i = 0; i < years.length; i++)
    years1[i] = 1960 + i;
yearBox1 = new JComboBox<>(years);
yearBox1.setBounds(280, 160, 80, 25);
PremiumPanel.add(yearBox1);

JLabel Personal = new JLabel("Personal Trainer:");
Personal.setBounds(385, 160, 120, 25);
PremiumPanel.add(Personal);
```

```
PersonalTrainer = new JTextField();
PersonalTrainer.setBounds(490, 160, 90, 25);
PremiumPanel.add(PersonalTrainer);

// Membership Start Date
JLabel msdLabel1 = new JLabel("Membership Start Date:");
msdLabel1.setBounds(20, 200, 180, 25);
PremiumPanel.add(msdLabel1);

msdDayBox1 = new JComboBox<>(days);
msdDayBox1.setBounds(200, 200, 50, 25);
PremiumPanel.add(msdDayBox1);

msdMonthBox1 = new JComboBox<>(months1);
msdMonthBox1.setBounds(260, 200, 100, 25);
PremiumPanel.add(msdMonthBox1);

msdYearBox1 = new JComboBox<>(years1);
msdYearBox1.setBounds(370, 200, 80, 25);
PremiumPanel.add(msdYearBox1);

// Add Member Button
addPremium2 = new JButton("Add Premium Member");
addPremium2.setBounds(20, 250, 190, 30);
addPremium2.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            addPremiumMember();
        }
    });
PremiumPanel.add(addPremium2);
```

```
// Clear Button
PremiumClearButton = new JButton("Clear");
PremiumClearButton.setBounds(270, 250, 95, 30);
PremiumClearButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        ID1.setText("");
        Name1.setText("");
        Location1.setText("");
        Phone1.setText("");
        Email1.setText("");
        PersonalTrainer.setText("");
        radioButtonn1.setSelected(false);
        radioButtonn2.setSelected(false);
        radioButtonn3.setSelected(false);

        dayBox1.setSelectedIndex(0);
        monthBox1.setSelectedIndex(0);
        yearBox1.setSelectedIndex(0);
        msdDayBox1.setSelectedIndex(0);
        msdMonthBox1.setSelectedIndex(0);
        msdYearBox1.setSelectedIndex(0);
    }
});
PremiumPanel.add(PremiumClearButton);

backPremiumButton = new JButton("Back");
backPremiumButton.setBounds(380, 250, 95, 30);
backPremiumButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
```

```
                PremiumFrame.setVisible(false);  // Hide the premium frame
                GymFrame.setVisible(true);      // Show the main frame
            }
        });
    PremiumPanel.add(backPremiumButton);

    //Active and Deactive for premium

    // Active and Deactive for premium
    // Active and Deactive for premium
    JLabel activateLabel1 = new JLabel("Member ID:");
    activateLabel1.setBounds(20, 300, 130, 25);
    PremiumPanel.add(activateLabel1);

    activatePremiumIdField = new JTextField();  // New separate field for premium
    activatePremiumIdField.setBounds(100, 300, 100, 25);
    PremiumPanel.add(activatePremiumIdField);

    activateBtn1 = new JButton("Activate");
    activateBtn1.setBounds(220, 300, 180, 25);
    activateBtn1.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            String inputId1 = activatePremiumIdField.getText().trim();  // Use the new
field
            try {
                int memberId1 = Integer.parseInt(inputId1);
                Gym_member member1 = findMemberById(memberId1);
                if (member1 != null) {
                    member1.activateMembership();
                    JOptionPane.showMessageDialog(PremiumFrame,
                        "Membership Activated for Member ID: " + memberId1);
```

```
                } else {
                    JOptionPane.showMessageDialog(PremiumFrame,
                        "Member ID not found.");
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Invalid Member ID entered.");
            }
        }
    });
PremiumPanel.add(activateBtn1);

// Deactivate Membership
JLabel deactivateLabel1 = new JLabel("Member ID:");
deactivateLabel1.setBounds(20, 340, 80, 25);
PremiumPanel.add(deactivateLabel1);

deactivateIdField1 = new JTextField();  // Changed from deactivateBtn2
deactivateIdField1.setBounds(100, 340, 100, 25);
PremiumPanel.add(deactivateIdField1);

// In your Premium Frame section, replace the deactivate button code with this:

deactivateBtn1 = new JButton("Deactivate");
deactivateBtn1.setBounds(220, 340, 180, 25);
deactivateBtn1.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String inputId1 = deactivateIdField1.getText().trim();
        try {
            int memberId1 = Integer.parseInt(inputId1);
            Gym_member member1 = findMemberById(memberId1);
```

```
                if (member1 == null) {
                    JOptionPane.showMessageDialog(PremiumFrame,
                        "Member ID " + memberId1 + " not found in system.",
                        "Not Found",
                        JOptionPane.WARNING_MESSAGE);
                }
                else if (!(member1 instanceof Premium_member)) {
                    JOptionPane.showMessageDialog(PremiumFrame,
                        "Member ID " + memberId1 + " is not a Premium Member.",
                        "Wrong Member Type",
                        JOptionPane.WARNING_MESSAGE);
                }
                else {
                    if (member1.get_ActiveStatus()) {
                        member1.deactiveMembership();
                        JOptionPane.showMessageDialog(PremiumFrame,
                            "Membership Deactivated for Premium Member ID: " +
memberId1,
                            "Success",
                            JOptionPane.INFORMATION_MESSAGE);
                    } else {
                        JOptionPane.showMessageDialog(PremiumFrame,
                            "Membership is already inactive for ID: " + memberId1,
                            "Already Inactive",
                            JOptionPane.WARNING_MESSAGE);
                    }
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Please enter a valid numeric Member ID",
```

```
                    "Invalid Input",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    });
PremiumPanel.add(deactivateBtn1);;

JLabel markAttendanceLabel1 = new JLabel("Member ID:");
markAttendanceLabel1.setBounds(20, 380, 80, 25);
PremiumPanel.add(markAttendanceLabel1);

markAttendanceIdField1 = new JTextField();
markAttendanceIdField1.setBounds(100, 380, 100, 25);
PremiumPanel.add(markAttendanceIdField1);
markAttendanceBtn1 = new JButton("Mark Attendance");
markAttendanceBtn1.setBounds(220, 380, 150, 25);
markAttendanceBtn1.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            String inputId = markAttendanceIdField1.getText().trim();
            try {
                int memberId = Integer.parseInt(inputId);
                Gym_member member = findMemberById(memberId);

                if (member == null) {
                    JOptionPane.showMessageDialog(PremiumFrame,
                        "Member ID " + memberId + " not found",
                        "Not Found",
                        JOptionPane.WARNING_MESSAGE);
                }
                else if (!(member instanceof Premium_member)) {
                    JOptionPane.showMessageDialog(PremiumFrame,
```

```
                    "Member ID " + memberId + " is not a Premium Member",
                    "Wrong Member Type",
                    JOptionPane.WARNING_MESSAGE);
                }
                else if (!member.get_ActiveStatus()) {
                    JOptionPane.showMessageDialog(PremiumFrame,
                    "Cannot mark attendance - membership is inactive",
                    "Inactive Membership",
                    JOptionPane.WARNING_MESSAGE);
                }
                else {
                    member.markAttendance();
                    JOptionPane.showMessageDialog(PremiumFrame,
                    "Attendance marked for Premium Member ID: " + memberId +
                    "\nTotal Attendance: " + member.get_Attendance() +
                    "\nLoyalty Points: " + member.get_LoyalPoints(),
                    "Attendance Recorded",
                    JOptionPane.INFORMATION_MESSAGE);
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(PremiumFrame,
                "Please enter a valid numeric Member ID",
                "Invalid Input",
                JOptionPane.ERROR_MESSAGE);
            }
        }
    });
PremiumPanel.add(markAttendanceBtn1);

JLabel revertPremiumLabel = new JLabel("Member ID:");
revertPremiumLabel.setBounds(20, 420, 80, 25);  // Adjusted position
```

```
        PremiumPanel.add(revertPremiumLabel);

        revertPremiumIdField = new JTextField();
        revertPremiumIdField.setBounds(100, 420, 100, 25);
        PremiumPanel.add(revertPremiumIdField);
        revertPremiumBtn = new JButton("Revert Premium Member");
        revertPremiumBtn.setBounds(220, 420, 180, 25);
        revertPremiumBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String inputId = revertPremiumIdField.getText().trim();
                try {
                    int memberId = Integer.parseInt(inputId);
                    Gym_member member = findMemberById(memberId);

                    if (member == null) {
                        JOptionPane.showMessageDialog(PremiumFrame,
                            "Member ID " + memberId + " not found",
                            "Not Found",
                            JOptionPane.WARNING_MESSAGE);
                    }
                    else if (!(member instanceof Premium_member)) {
                        JOptionPane.showMessageDialog(PremiumFrame,
                            "Member ID " + memberId + " is not a Premium Member",
                            "Wrong Member Type",
                            JOptionPane.WARNING_MESSAGE);
                    }
                    else {
                        ((Premium_member)member).revertPremiumMember();
                        JOptionPane.showMessageDialog(PremiumFrame,
                            "Premium Member ID " + memberId + " has been reverted\n" +
                            "All data has been reset to default values",
```

```
                    "Premium Member Reverted",
                    JOptionPane.INFORMATION_MESSAGE);
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Please enter a valid numeric Member ID",
                    "Invalid Input",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    });
PremiumPanel.add(revertPremiumBtn);

// Calculate Discount Section
JLabel discountLabel = new JLabel("Member ID:");
discountLabel.setBounds(20, 460, 80, 25);
PremiumPanel.add(discountLabel);

JTextField discountIdField = new JTextField();
discountIdField.setBounds(100, 460, 100, 25);
PremiumPanel.add(discountIdField);

JButton calculateDiscountBtn = new JButton("Calculate Discount");
calculateDiscountBtn.setBounds(220, 460, 180, 25);
calculateDiscountBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            String inputId = discountIdField.getText().trim();
            try {
                int memberId = Integer.parseInt(inputId);
                Gym_member member = findMemberById(memberId);
```

```
                if (member != null && member instanceof Premium_member) {
                    ((Premium_member)member).calculateDiscount();
                    JOptionPane.showMessageDialog(PremiumFrame,
                        "Discount calculated for Premium Member ID: " + memberId +
                        "\nDiscount Amount: Rs. " +
((Premium_member)member).get_discountAmount(),
                        "Discount Calculation",
                        JOptionPane.INFORMATION_MESSAGE);
                } else {
                    JOptionPane.showMessageDialog(PremiumFrame,
                        "Premium Member ID not found",
                        "Not Found",
                        JOptionPane.WARNING_MESSAGE);
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Invalid Member ID format",
                    "Input Error",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    });
    PremiumPanel.add(calculateDiscountBtn);

    // Pay Due Amount Section
    JLabel paymentLabel = new JLabel("Amount:");
    paymentLabel.setBounds(20, 500, 80, 25);
    PremiumPanel.add(paymentLabel);

    JTextField amountField = new JTextField();
    amountField.setBounds(100, 500, 100, 25);
```

```
PremiumPanel.add(amountField);

JButton payAmountBtn = new JButton("Pay Due Amount");
payAmountBtn.setBounds(220, 500, 180, 25);
payAmountBtn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String inputId = discountIdField.getText().trim();
        String amountStr = amountField.getText().trim();

        try {
            int memberId = Integer.parseInt(inputId);
            double amount = Double.parseDouble(amountStr);

            if (amount <= 0) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Payment amount must be positive",
                    "Invalid Amount",
                    JOptionPane.WARNING_MESSAGE);
                return;
            }

            Gym_member member = findMemberById(memberId);

            if (member == null) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Member ID " + memberId + " not found",
                    "Not Found",
                    JOptionPane.WARNING_MESSAGE);
            }
            else if (!(member instanceof Premium_member)) {
                JOptionPane.showMessageDialog(PremiumFrame,
```

```
                    "Member ID " + memberId + " is not a Premium Member",
                    "Wrong Member Type",
                    JOptionPane.WARNING_MESSAGE);
                }
                else {
                    Premium_member pm = (Premium_member)member;
                    String result = pm.payDueAmount(amount);

                    JOptionPane.showMessageDialog(PremiumFrame,
                        "Payment for Member ID: " + memberId + "\n" +
                        "Premium Charge: Rs. " + pm.get_premiumCharge() + "\n" +
                        "Amount Paid: Rs. " + amount + "\n" +
                        result + "\n" +
                        "Remaining Balance: Rs. " + (pm.get_premiumCharge() -
pm.get_paidAmount()),
                        "Payment Status",
                        JOptionPane.INFORMATION_MESSAGE);

                    amountField.setText("");
                }
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Please enter valid numbers for both fields",
                    "Input Error",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    });
    PremiumPanel.add(payAmountBtn);

    // Display All Premium Members Button
```

```
        JButton displayAllBtn = new JButton("Display All Premium Members");
        displayAllBtn.setBounds(20, 540, 380, 25);
        displayAllBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                StringBuilder allMembersInfo = new StringBuilder();
                int premiumCount = 0;

                for (Gym_member member : arr) {
                    if (member instanceof Premium_member) {
                        Premium_member pm = (Premium_member) member;
                        allMembersInfo.append("ID: ").append(pm.get_id()).append("\n")
                        .append("Name: ").append(pm.get_name()).append("\n")
                        .append("Personal Trainer:
").append(pm.get_personalTrainer()).append("\n")
                        .append("Paid Amount: Rs.
").append(pm.get_paidAmount()).append("\n")
                        .append("Full Payment: ").append(pm.get_isFullPayment() ? "Yes" :
"No").append("\n");
                        premiumCount++;
                    }
                }

                if (premiumCount > 0) {
                    JOptionPane.showMessageDialog(PremiumFrame,
                        allMembersInfo.toString(),
                        "All Premium Members (" + premiumCount + ")",
                        JOptionPane.INFORMATION_MESSAGE);
                } else {
                    JOptionPane.showMessageDialog(PremiumFrame,
                        "No premium members found",
                        "Information",
```

```
                    JOptionPane.INFORMATION_MESSAGE);
                }
            }
        });
    PremiumPanel.add(displayAllBtn);

    //save file
    JButton saveToFileBtn = new JButton("Save to File");
    saveToFileBtn.setBounds(20, 590, 150, 25);
    saveToFileBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                save(arr);  // Call the save method with the member list
            }
        });
    //read file
    JButton readFromFileBtn = new JButton("Read from File");
    readFromFileBtn.setBounds(200, 590, 150, 25);
    readFromFileBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                read();  // Call the read method
            }
        });

    // Add them to your panel
    PremiumPanel.add(saveToFileBtn);
    PremiumPanel.add(readFromFileBtn);
    // Add Premium Panel to Frame
    PremiumFrame.getContentPane().add(PremiumPanel);
}

public Gym_member findMemberById(int id) {
```

```
        for (Gym_member member : arr) {
            if (member.get_id() == id) {
                return member;
            }
        }
        return null;
    }

    public void addRegularMember() {
        try {
            int id = Integer.parseInt(this.ID.getText().trim());
            if (findMemberById(id) != null) {
                JOptionPane.showMessageDialog(RegularFrame,
                    "Member ID already exists!",
                    "Duplicate ID",
                    JOptionPane.WARNING_MESSAGE);
                return;
            }

            String name = this.Name.getText().trim();
            String location = this.Location.getText().trim();
            String phone = this.Phone.getText().trim();
            String email = this.Email.getText().trim();
            String referralSource = this.ReferralSource.getText().trim();
            String plan = (String) planBox.getSelectedItem();

            if (name.isEmpty() || location.isEmpty() || phone.isEmpty() ||
            email.isEmpty() || referralSource.isEmpty()) {
                JOptionPane.showMessageDialog(RegularFrame,
                    "All fields must be filled out!",
                    "Input Error",
```

```
            JOptionPane.WARNING_MESSAGE);
        return;
    }

    if (!email.matches("^[\\w-.]+@([\\w-]+\\.)+[\\w-]{2,4}$")) {
        JOptionPane.showMessageDialog(RegularFrame,
            "Please enter a valid email address",
            "Invalid Email",
            JOptionPane.WARNING_MESSAGE);
        return;
    }

    if (!phone.matches("^[0-9]{10,15}$")) {
        JOptionPane.showMessageDialog(RegularFrame,
            "Phone number must be 10-15 digits",
            "Invalid Phone",
            JOptionPane.WARNING_MESSAGE);
        return;
    }

    String gender = this.radioButton1.isSelected() ? "Male" :
        this.radioButton2.isSelected() ? "Female" : "Other";

    int day = (Integer) dayBox.getSelectedItem();
    String month = (String) monthBox.getSelectedItem();
    int year = (Integer) yearBox.getSelectedItem();
    String DOB = day + " " + month + " " + year;

    int msdDay = (Integer) msdDayBox.getSelectedItem();
    String msdMonth = (String) msdMonthBox.getSelectedItem();
    int msdYear = (Integer) msdYearBox.getSelectedItem();
```

```
            String MembershipStartDate = msdDay + " " + msdMonth + " " + msdYear;

            Regular_member regularMember = new Regular_member( id,name, location,
phone, email,gender,
                    DOB,MembershipStartDate,referralSource);
            arr.add(regularMember);

            JOptionPane.showMessageDialog(RegularFrame,
                "Regular Member Added Successfully!\n" +
                "ID: " + id + "\n" +
                "Name: " + name + "\n" +
                "Plan: " + plan,
                "Success",
                JOptionPane.INFORMATION_MESSAGE);

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(RegularFrame,
                "ID must be a valid number!",
                "Input Error",
                JOptionPane.ERROR_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(RegularFrame,
                "Error adding member: " + ex.getMessage(),
                "Error",
                JOptionPane.ERROR_MESSAGE);
            ex.printStackTrace();
        }
    }

    public void addPremiumMember() {
        try {
```

```
        // Validate numeric ID first
        int id = Integer.parseInt(this.ID1.getText().trim());

        // Validate required text fields
        String name = this.Name1.getText().trim();
        String location = this.Location1.getText().trim();
        String phone = this.Phone1.getText().trim();
        String email = this.Email1.getText().trim();
        String trainer = this.PersonalTrainer.getText().trim();

        if (name.isEmpty() || location.isEmpty() ||  trainer.isEmpty()) {
            JOptionPane.showMessageDialog(PremiumFrame,
                "All fields must be filled out!",
                "Input Error",
                JOptionPane.WARNING_MESSAGE);
            return;
        }

        // Get gender selection
        String gender = this.radioButtonn1.isSelected() ? "Male" :
            this.radioButtonn2.isSelected() ? "Female" : "Other";

        // Get dates from combo boxes (no parsing needed)
        int day = (Integer) dayBox1.getSelectedItem();
        String month = (String) monthBox1.getSelectedItem();
        int year = (Integer) yearBox1.getSelectedItem();
        String DOB = day + " " + month + " " + year;

        int msdDay = (Integer) msdDayBox1.getSelectedItem();
        String msdMonth = (String) msdMonthBox1.getSelectedItem();
        int msdYear = (Integer) msdYearBox1.getSelectedItem();
```

```
            String MembershipStartDate = msdDay + " " + msdMonth + " " + msdYear;
            if (!email.matches("^[\\w-.]+@([\\w-]+\\.)+[\\w-]{2,4}$")) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Please enter a valid email address",
                    "Invalid Email",
                    JOptionPane.WARNING_MESSAGE);
                return;
            }
            if (!phone.matches("^[0-9]{10,15}$")) {
                JOptionPane.showMessageDialog(PremiumFrame,
                    "Phone number must be 10-15 digits",
                    "Invalid Phone",
                    JOptionPane.WARNING_MESSAGE);
                return;
            }

            // Create and add member (include the ID in constructor)
            Premium_member premiumMember = new Premium_member(id, name,
location, phone, email,
                    gender, DOB, MembershipStartDate, trainer);
            arr.add(premiumMember);

            JOptionPane.showMessageDialog(PremiumFrame,
                "Premium Member Added Successfully",
                "Success",
                JOptionPane.INFORMATION_MESSAGE);

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(PremiumFrame,
                "ID must be a valid number!",
                "Input Error",
```

```
                JOptionPane.ERROR_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(PremiumFrame,
                "Error adding member: " + ex.getMessage(),
                "Error",
                JOptionPane.ERROR_MESSAGE);
        }

    }

    public void showRegularFrame() {
        RegularFrame.setVisible(true);
    }

    public void showPremiumFrame() {
        PremiumFrame.setVisible(true);
    }

    public void save(ArrayList<Gym_member> arr)
    {
        File file = new File("MemberDetails.txt");
        try
        {
            if(!file.exists())
            {
                file.createNewFile();
            }
            try(FileWriter writer = new FileWriter(file))
            {
                if(file.length() == 0)
                {
```

```
            String header = String.format("%-4s %-15s %-10s %-12s %-22s %-7s %-
10s %-8s %-19s %-21s %-13s %-17s %-15s %-11s %-13s %-13s%n",
                    "ID","Name", "Location", "Phone", "Email","Gender", "Trainer's name",
"Plan", "Date of birth", "Membership start date", "Attendance", "Loyalty points",
                    "Active status", "Discount", "Full payment", "Paid Amount" );

            String line = "----------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------------------
---------------------------------------;----------------------------------------------------------------------
-------------------------------------------------------------------------------------\n";
               writer.write(header);
               writer.write(line);
            }
            for(Gym_member member : arr)
            {
               if(member instanceof Regular_member)
               {
                  Regular_member regularMember = (Regular_member) member;

                  String line = String.format("%-4s %-15s %-12s %-12s %-22s %-7s %-
15s %-8s %-19s %-21s %-13s %-17s %-15s %-11s %-13s %-13s%n",
                        regularMember.get_id(),
                        regularMember.get_name(),
                        regularMember.get_Location(),
                        regularMember.get_Phone(),
                        regularMember.get_email(),
                        regularMember.get_Gender(),
                        "-",
                        regularMember.get_Plan(),
                        regularMember.get_DOB(),
```

```
                regularMember.get_MembershipStartDate(),
                regularMember.get_Attendance(),
                regularMember.get_LoyalPoints(),
                regularMember.get_ActiveStatus(),
                "-",
                "-",
                "-");
            writer.write(line);
        }
        else if(member instanceof Premium_member)
        {
            Premium_member premiumMember = (Premium_member) member;

            String line = String.format("%-4s %-15s %-12s %-12s %-22s %-7s %-
15s %-8s %-19s %-21s %-13s %-17s %-15s %-11s %-13s %-13s%n",
                premiumMember.get_id(),
                premiumMember.get_name(),
                premiumMember.get_Location(),
                premiumMember.get_Phone(),
                premiumMember.get_email(),
                premiumMember.get_Gender(),
                premiumMember.get_personalTrainer(),
                "-",
                premiumMember.get_DOB(),
                premiumMember.get_MembershipStartDate(),
                premiumMember.get_Attendance(),
                premiumMember.get_LoyalPoints(),
                premiumMember.get_ActiveStatus(),
                premiumMember.get_discountAmount(),
                premiumMember.get_isFullPayment(),
                premiumMember.get_paidAmount());
```

```
                writer.write(line);
            }
        }
            JOptionPane.showMessageDialog(Name, "Data stored successfully!",
"Successful!", JOptionPane.INFORMATION_MESSAGE);
        }
    }
    catch(IOException e)
    {
        JOptionPane.showMessageDialog(Name, "Unexpected error!", "Error!",
JOptionPane.ERROR_MESSAGE);
    }
  }

  public void read()
  {
    java.util.List<String> lines = new ArrayList<>();
    String line;

    File file = new File("MemberDetails.txt");
    try(BufferedReader reader = new BufferedReader(new FileReader(file)))
    {
      while ((line = reader.readLine()) != null)
      {
        lines.add(line);
      }
    }
    catch(IOException e)
    {
      System.out.println(e.getMessage());
```

```
        }
        JFrame frame = new JFrame("Read");
        frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        frame.setSize(1540, 500);
        frame.setLayout(null);
        frame.setResizable(true);
        int y = 5;
        for (String text : lines)
        {
            JLabel label = new JLabel(text);
            label.setFont(new Font("monospaced", Font.PLAIN, 10));
            label.setBounds(10, y, 1540, 20);
            frame.add(label);
            y+=25;
        }
        frame.setVisible(true);
    }
}

class RegularButtonListener implements ActionListener {
    private Gym_GUI gui;

    public RegularButtonListener(Gym_GUI gui) {
        this.gui = gui;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        gui.showRegularFrame();
    }
}
```

```
class PremiumButtonListener implements ActionListener {
    private Gym_GUI gui;

    public PremiumButtonListener(Gym_GUI gui) {
        this.gui = gui;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        gui.showPremiumFrame();
    }
}
```

Premium_member

```
/**
 * Write a description of class Premium_member here.
 *The Premium_member class extends Gym_member and represents a premium gym
member who has additional features like a personaltrainer,
  full payment status, and premium charges. It includes methods for marking
attendance, paying dues, calculating discounts for full
  payments, and reverting member details. The class tracks payment amounts,
calculates remaining dues, and displays detailed
  information such as the personal trainer and discount amount.
 * @author (Priya_Thapa)
 * @version (28/02/2025)
 */
class Premium_member extends Gym_member
{
    private final double premiumCharge;
    private String personalTrainer;
```

```
    private boolean isFullPayment;
    private double paidAmount;
    private double discountAmount;
    public Premium_member(int id1,String Name1,String Location1,String Phone1,String
email1,String Gender1,
   String DOB1,String MembershipStartDate1, String personalTrainer1){
       super(id1, Name1, Location1, Phone1, email1, Gender1,DOB1,
MembershipStartDate1);
       this.premiumCharge=50000.0;
       this.paidAmount=0.0;
       this.isFullPayment=false;
       this.discountAmount=0.0;
       this.personalTrainer=personalTrainer1;
   }
   public double get_premiumCharge(){
       return this.premiumCharge;
   }
     public double get_paidAmount(){
       return this.paidAmount;
   }
   public boolean get_isFullPayment(){
        return this.isFullPayment;
   }
     public double get_discountAmount(){
       return this.discountAmount;
   }
     public String get_personalTrainer(){
       return this.personalTrainer;
   }

   public void markAttendance(){
```

```
        if(super.ActiveStatus==true){
        super.Attendance++;
        super.LoyalPoints +=10.0;
        System.out.println("Attendance marked for Premium Member: " + super.Name);
    System.out.println("Total Attendance: " + super.Attendance);
    System.out.println("Loyalty Points: " +super.LoyalPoints);
}
else{
    System.out.println("Account is deactive" );
}
}


   /*Method to handle payment of dues and check full payment status*/
   public String payDueAmount(double paidAmount) {
    this.paidAmount = paidAmount;

    if(this.isFullPayment) {
        return "Amount had been paid already";
    }
    else if(this.premiumCharge < paidAmount) {
        this.isFullPayment = true;
        return "Amount exceeds premium charge. Full payment recorded.";
    }
    else if(this.premiumCharge > paidAmount) {
        double remainingAmount = this.premiumCharge - paidAmount;
        this.isFullPayment = false;
        return "Remaining amount to pay: Rs. " + remainingAmount;
    }
    else {
        this.isFullPayment = true;
```

```
        return "Full payment received";
    }
}
   /*Method to calculate discount for full payment*/
   public void calculateDiscount(){
     if(this.isFullPayment==true){
         this.discountAmount=0.1*this.premiumCharge;
        System.out.println("Your discount is: " +this.discountAmount);
     }
     else{
         System.out.println("No discount");
     }
   }

   public void revertPremiumMember()
    {
        super.resetMember();
        this.personalTrainer="";
        this.isFullPayment=false;
        this.paidAmount=0.0;
        this.discountAmount=0.0;
    }

   public void display(){
    super.display(); /*Display basic member details*/
    System.out.println("Personal Trainer : " + this.personalTrainer);
    System.out.println("Paid Amount : " + this.paidAmount);
    System.out.println("Full Payment : " + this.isFullPayment);
    double remainingAmount=this.premiumCharge-paidAmount;
    System.out.println("Remaining Amount : " + remainingAmount);
    if(isFullPayment == true)
```

```
     {
     System.out.println("Discount Amount : " + this.discountAmount);
     }
   }
    }
```

Gym_member

```
/**
 *The Gym_member class is an abstract class representing a gym member's details and membership status. It stores personal information
such as name, contact details, and membership data like attendance and loyalty points, while providing methods to activate/
deactivate memberships, reset member data, and display their information.
 * @author (Priya Thapa)
 * @version (25/2/2025)
 */
abstract class Gym_member
{
    protected int id;
    protected String Name;
    protected String Location;
    protected String Phone;
    protected String email;
    protected String Gender;
    protected String DOB;
    protected String MembershipStartDate;
    protected int Attendance;
    protected double LoyalPoints;
    protected boolean ActiveStatus;
```

```
    /*Constructor to initialize the gym member's details*/
    public Gym_member(int id1,String Name1,String Location1,String Phone1,String
email1,String Gender1,
    String DOB1,String MembershipStartDate1){
        this.Attendance=0;
        this.ActiveStatus=false;
        this.LoyalPoints=0.0;
        this.id=id1;
        this.Name=Name1;
        this.Location=Location1;
        this.Phone=Phone1;
        this.email=email1;
        this.Gender=Gender1;
        this.DOB=DOB1;
        this.MembershipStartDate=MembershipStartDate1;

    }

    /* Getter methods for all the attributes (to access values of private fields)*/
    public int get_id(){
        return this.id;
    }

    public String get_name(){
        return this.Name;
    }

    public String get_Location(){
        return this.Location;
    }
```

```
    public String get_Phone(){
        return this.Phone;
    }

    public String get_email(){
        return this.email;
    }

    public String get_Gender(){
        return this.Gender;
    }

    public String get_DOB(){
        return this.DOB;
    }

    public String get_MembershipStartDate(){
        return this.MembershipStartDate;
    }

    public int get_Attendance()
    {
        return this.Attendance;
    }

    public boolean get_ActiveStatus()
    {
        return this.ActiveStatus;
    }

    public double get_LoyalPoints() {
```

```
        return this.LoyalPoints;
    }

    public abstract  void markAttendance ();

    /* Method to activate the membership */
    public void activateMembership() {
        if(this.ActiveStatus==false){
            this.ActiveStatus=true;
            System.out.println("The membership"+this.id+" is successfully activated");
        }
        else
        {
            System.out.println("The membership"+this.id+" is already activated");
        }
    }

    /* Method to deactivate the membership if it's active*/
    public void deactiveMembership() {
        if(this.ActiveStatus==true){
            this.ActiveStatus=false;
            System.out.println("The membership"+this.id+" is successfully deactivated");
        }
        else{
            System.out.println("The membership"+this.id+" is already deactivated");
        }
    }

    /* Method to reset the members data */
    public void resetMember()
    {
```

```
        this.ActiveStatus=false;
        this.Attendance=0;
        this.LoyalPoints=0.0;
        System.out.println("Membership"+this.id+"is reset successfully");
    }

    /* Method to display the member's details*/
    public void display(){
        System.out.println("Member ID:"+get_id());
        System.out.println("Member Name:"+get_name());
        System.out.println("Member Location:"+get_Location());
        System.out.println("Member phone number:"+get_Phone());
        System.out.println("Member Email:"+get_email());
        System.out.println("Gender of member:"+get_Gender());
        System.out.println("Date of birth of member"+get_DOB());
        System.out.println("Membership start date"+get_MembershipStartDate());
        System.out.println("Membership attendace"+get_Attendance());
        System.out.println("Membership loyal point"+get_LoyalPoints());
        System.out.println("Membership active status"+get_ActiveStatus());
    }
}
```

Regular_member

```
/**
 * Write a description of class Regular_member here.
 *The Regular_member class extends Gym_member and represents a regular gym
member with features like attendance tracking, loyalty
points, and plan upgrades. It includes methods to mark attendance, check eligibility for
plan upgrades, update plan prices, and
```

revert the membership details. The class also manages member-specific attributes like attendance limit, plan, price, and referral
source.
 * @author (Priya_Thapa)
 * @version (26/02/2025)
 */
class Regular_member extends Gym_member{
    private final int attendanceLimit;
    private boolean isEligibleForUpgrade;
    private String removalReason;
    private String referralSource;
    private String Plan;
    private double Price;
    public Regular_member(int id1,String Name1,String Location1,String Phone1,String email1,String Gender1,
    String DOB1,String MembershipStartDate1,String referralSource1){
        super(id1, Name1, Location1, Phone1, email1, Gender1,DOB1, MembershipStartDate1);
        this.isEligibleForUpgrade=false;
        this.attendanceLimit=30;
        this.Plan="Basic";
        this.Price=6500.0;
        this.removalReason="";
        this.referralSource=referralSource1;

    }

    public boolean get_isEligibleForUpgrade(){
        return this.isEligibleForUpgrade;
    }

```
    public int get_attendanceLimit(){
        return this.attendanceLimit;
    }

    public String get_Plan(){
        return this.Plan;
    }

    public double get_Price(){
        return this.Price;
    }

    public String get_removalReason(){
        return this.removalReason;
    }

    public String get_referralSource(){
        return this.referralSource;
    }

    public void markAttendance(){
        if(this.ActiveStatus==true){
            super.Attendance++;
            super.LoyalPoints +=5.0;
            System.out.println("Attendance marked for Regular Member: " + super.Name);
            System.out.println("Total Attendance: " + super.Attendance);
            System.out.println("Loyalty Points: " +super.LoyalPoints);
        }
        else{
            System.out.println("Account is deactive" );
        }
```

```
    }

    /* Method to get the price based on the selected plan*/
    public double getPlanPrice(String Plan){
        switch(Plan.toLowerCase()){
            case "basic":
                System.out.println("Price of this"+this.Plan+"plan is 6500");
                this.Price = 6500.0;
                return this.Price;

            case "standard":
                System.out.println("Price of this"+this.Plan+"plan is 12500");
                this.Price = 12500.0;
                return this.Price;

            case "deluxe":
                System.out.println("Price of this"+this.Plan+"plan is 18500");
                this.Price = 18500.0;
                return this.Price;

            default:
                return -1;

        }
    }

    public String upgradePlan(String newPlan) {
        // Check eligibility
        if (this.Attendance >= this.attendanceLimit) {
            this.isEligibleForUpgrade = true;
        } else {
```

```
            this.isEligibleForUpgrade = false;
        }

        if (!this.isEligibleForUpgrade) {
            return "You are not eligible for an upgrade yet.";
        }

        if (this.Plan.equalsIgnoreCase(newPlan)) {
            return "You are already subscribed to this plan.";
        }

        // Assign new plan first
        this.Plan = newPlan;

        // Get and update price
        double newPrice = getPlanPrice(newPlan);

        if (newPrice == -1) {
            return "Invalid plan selected.";
        }

        this.Price = newPrice;  // Correctly update price
        return "Plan upgraded successfully to " + newPlan + " with price " + newPrice;
    }

    public void revertRegularMember(String removalReason) {
        super.resetMember();  // Reset member details to default state
        this.isEligibleForUpgrade = false;
        this.Plan = "basic";
        this.Price = 6500;
        this.removalReason = removalReason;
```

```
    }

    public void display(){
        super.display();
        System.out.println("Plan: " + this.Plan);
        System.out.println("Price: " + this.Price);

        if (!this.removalReason.isEmpty()) {
            System.out.println("Removal Reason: " + this.removalReason);
        }
    }

}
```

this my code

```
/**
 * Write a description of class Premium_member here.
 *The Premium_member class extends Gym_member and represents a premium gym member who has additional features like a personaltrainer,
  full payment status, and premium charges. It includes methods for marking attendance, paying dues, calculating discounts for full
  payments, and reverting member details. The class tracks payment amounts, calculates remaining dues, and displays detailed
  information such as the personal trainer and discount amount.
 * @author (Priya_Thapa)
 * @version (28/02/2025)
 */
class Premium_member extends Gym_member
{
    private final double premiumCharge;
    private String personalTrainer;
    private boolean isFullPayment;
```

```
    private double paidAmount;
    private double discountAmount;
    public Premium_member(int id1,String Name1,String Location1,String Phone1,String
email1,String Gender1,
   String DOB1,String MembershipStartDate1, String personalTrainer1){
       super(id1, Name1, Location1, Phone1, email1, Gender1,DOB1,
MembershipStartDate1);
       this.premiumCharge=50000.0;
       this.paidAmount=0.0;
       this.isFullPayment=false;
       this.discountAmount=0.0;
       this.personalTrainer=personalTrainer1;
   }
   public double get_premiumCharge(){
       return this.premiumCharge;
   }
     public double get_paidAmount(){
       return this.paidAmount;
   }
   public boolean get_isFullPayment(){
        return this.isFullPayment;
   }
     public double get_discountAmount(){
       return this.discountAmount;
   }
     public String get_personalTrainer(){
       return this.personalTrainer;
   }

   public void markAttendance(){
       if(super.ActiveStatus==true){
```

```
        super.Attendance++;
        super.LoyalPoints +=10.0;
        System.out.println("Attendance marked for Premium Member: " + super.Name);
    System.out.println("Total Attendance: " + super.Attendance);
    System.out.println("Loyalty Points: " +super.LoyalPoints);
}
else{
    System.out.println("Account is deactive" );
}
}


   /*Method to handle payment of dues and check full payment status*/
   public String payDueAmount(double paidAmount) {
    this.paidAmount = paidAmount;

    if(this.isFullPayment) {
        return "Amount had been paid already";
    }
    else if(this.premiumCharge < paidAmount) {
        this.isFullPayment = true;
        return "Amount exceeds premium charge. Full payment recorded.";
    }
    else if(this.premiumCharge > paidAmount) {
        double remainingAmount = this.premiumCharge - paidAmount;
        this.isFullPayment = false;
        return "Remaining amount to pay: Rs. " + remainingAmount;
    }
    else {
        this.isFullPayment = true;
        return "Full payment received";
```

```
    }
}
   /*Method to calculate discount for full payment*/
   public void calculateDiscount(){
      if(this.isFullPayment==true){
          this.discountAmount=0.1*this.premiumCharge;
         System.out.println("Your discount is: " +this.discountAmount);
      }
      else{
          System.out.println("No discount");
      }
   }

   public void revertPremiumMember()
    {
        super.resetMember();
        this.personalTrainer="";
        this.isFullPayment=false;
        this.paidAmount=0.0;
        this.discountAmount=0.0;
    }

   public void display(){
    super.display(); /*Display basic member details*/
    System.out.println("Personal Trainer : " + this.personalTrainer);
    System.out.println("Paid Amount : " + this.paidAmount);
    System.out.println("Full Payment : " + this.isFullPayment);
    double remainingAmount=this.premiumCharge-paidAmount;
    System.out.println("Remaining Amount : " + remainingAmount);
    if(isFullPayment == true)
      {
```

```
    System.out.println("Discount Amount : " + this.discountAmount);
    }
  }
   }
```